MACHINE LEARNING (ML)
BTCSE023602

UNIT-III
SUPERVISED LEARNING

Mr. Naresh A. Kamble

Asst. Professor,

Department of Computer Science and Engineering

D.Y.Patil Agriculture and Technical University, Talsande, Kolhapur

COURSE OUTCOME

After end of this unit, students will be able to

CO2: Implement supervised learning algorithms.

CONTENTS

- **Regression Algorithms**
- **Classification Algorithms**
- **Model Evaluation**

REGRESSION

INTRODUCTION

- **REGRESSION**
- ***In regression problems, the output is a continuous variable.***

LINEAR REGRESSION ALGORITHM

LINEAR REGRESSION

- **INTRODUCTION**
- Linear regression is a type of **supervised machine learning algorithm** that computes the **linear relationship** between the **dependent variable** and **one** or **more independent features** by **fitting** a **linear equation** to **observed data**.

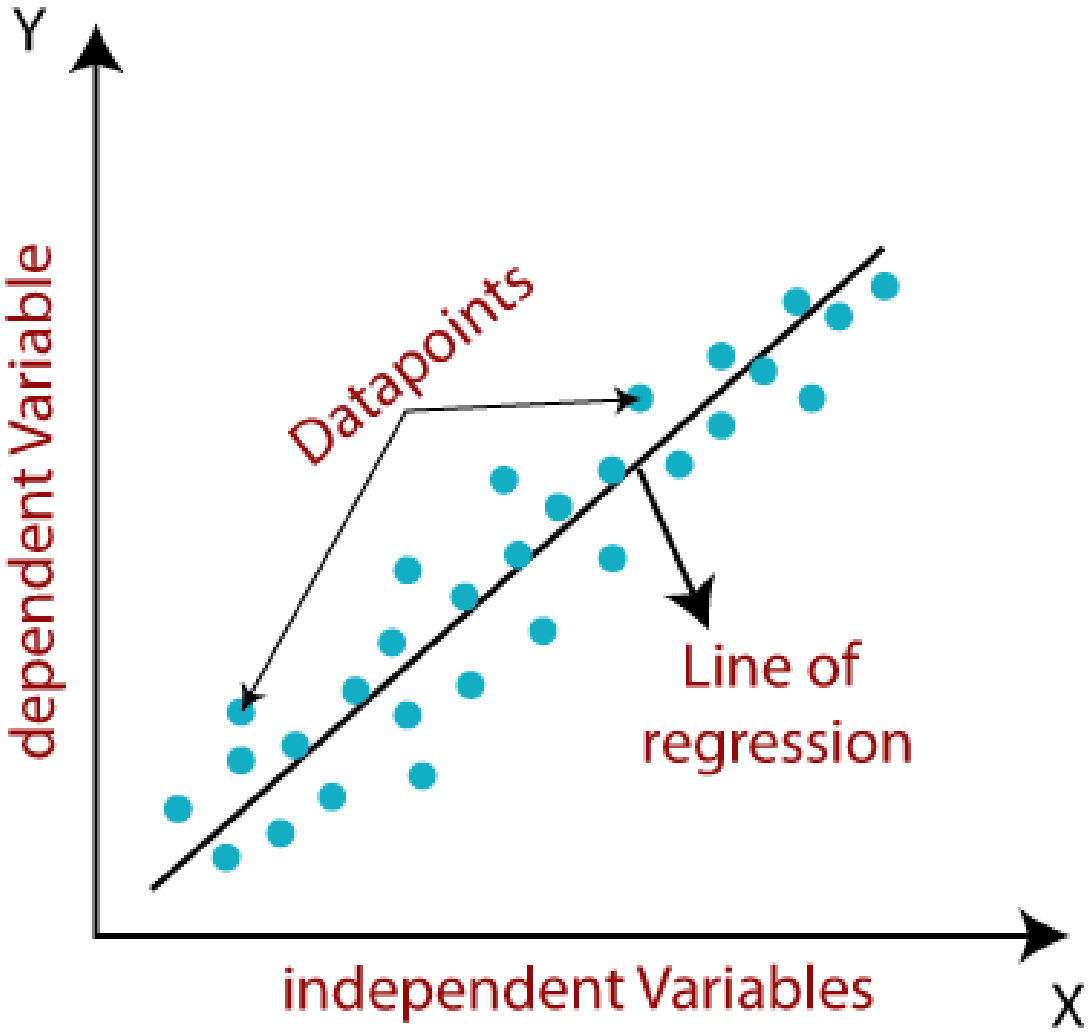
LINEAR REGRESSION

- **INTRODUCTION**
- It is represented in the form of a line:

$$\boldsymbol{y = mx + b}$$

- Where,
- **y** = dependent variable
- **x** = independent variable
- **m** = slope of line
- **b** = y intercept

LINEAR REGRESSION



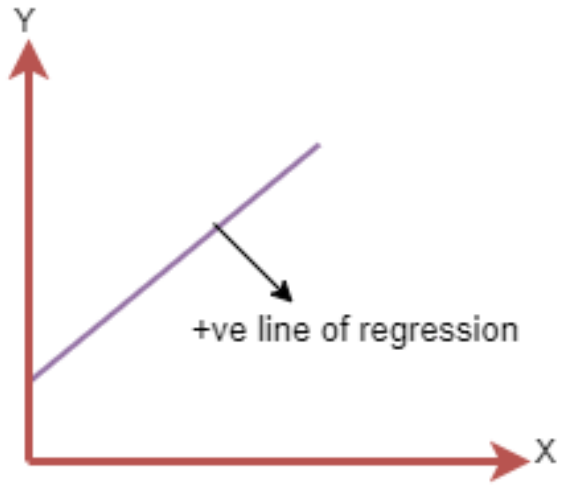
LINEAR REGRESSION

- **LINEAR REGRESSION LINE**
- ***A linear line showing the relationship between the dependent and independent variables is called a regression line.***

- A regression line can show two types of relationship:
- **Positive Linear Relationship**
- **Negative Linear Relationship**

LINEAR REGRESSION

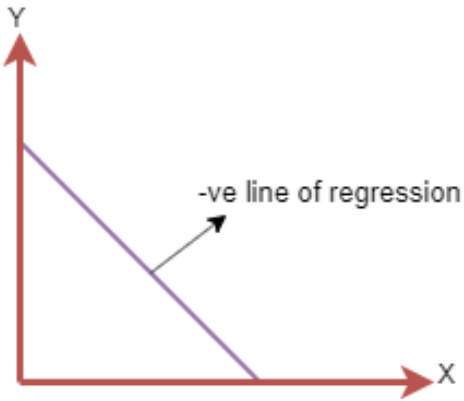
- **POSITIVE LINEAR RELATIONSHIP:**
- If the **dependent variable increases** on the **Y-axis** and **independent variable increases** on **X-axis**, then such a relationship is termed as a ***positive linear relationship***.



The line equation will be: $Y = a_0 + a_1 x$

LINEAR REGRESSION

- **NEGATIVE LINEAR RELATIONSHIP:**
- If the **dependent variable decreases** on the **Y-axis** and **independent variable increases** on the **X-axis**, then such a relationship is called a ***negative linear relationship***.



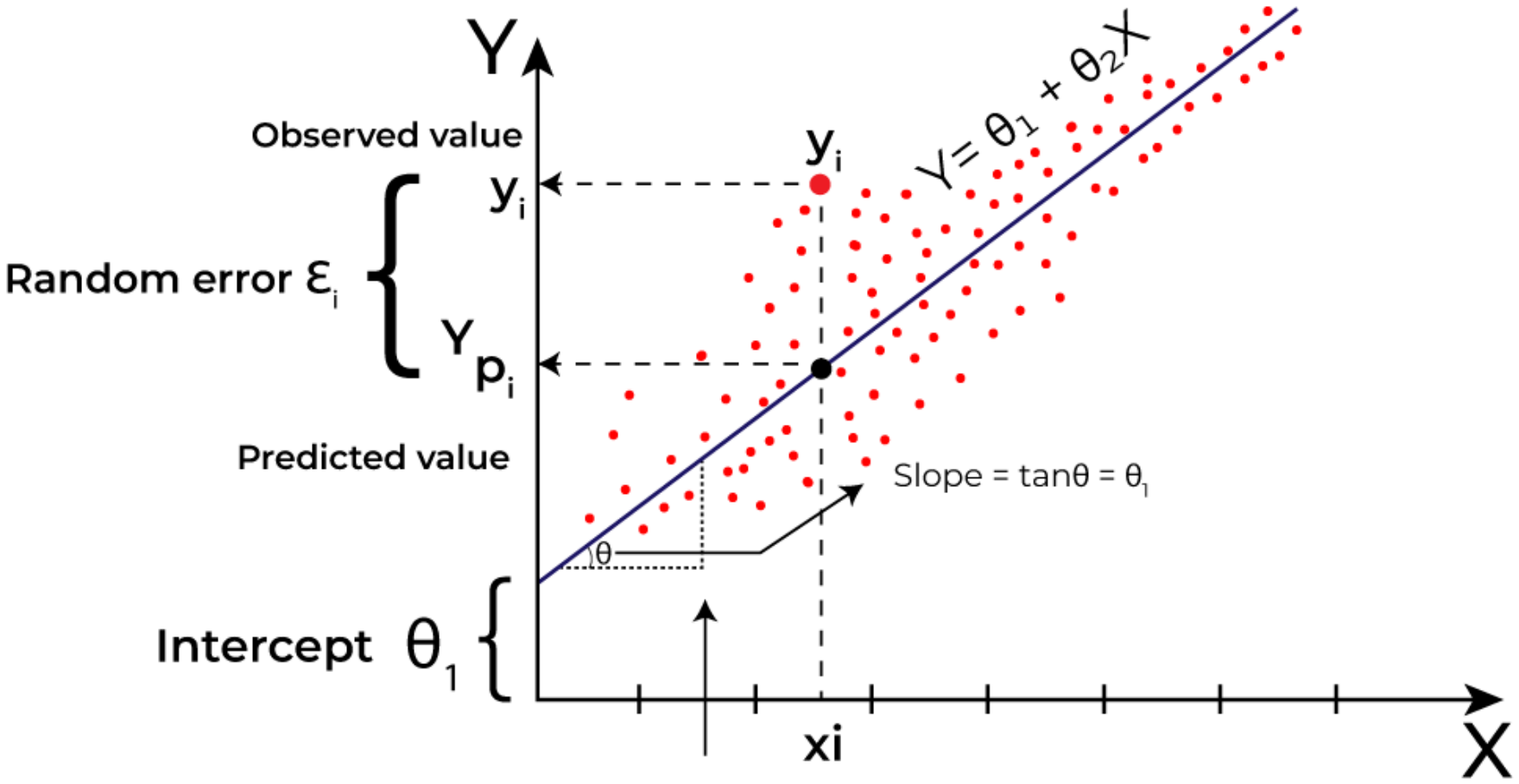
The line of equation will be: $Y= -a_0+a_1x$

LINEAR REGRESSION

- **BEST FIT LINE**
- Our main goal is to find the **best fit line** that means the **error** between **predicted values** and **actual values** should be minimized.

- The **best fit line** will have the **least error**.

LINEAR REGRESSION



LINEAR REGRESSION

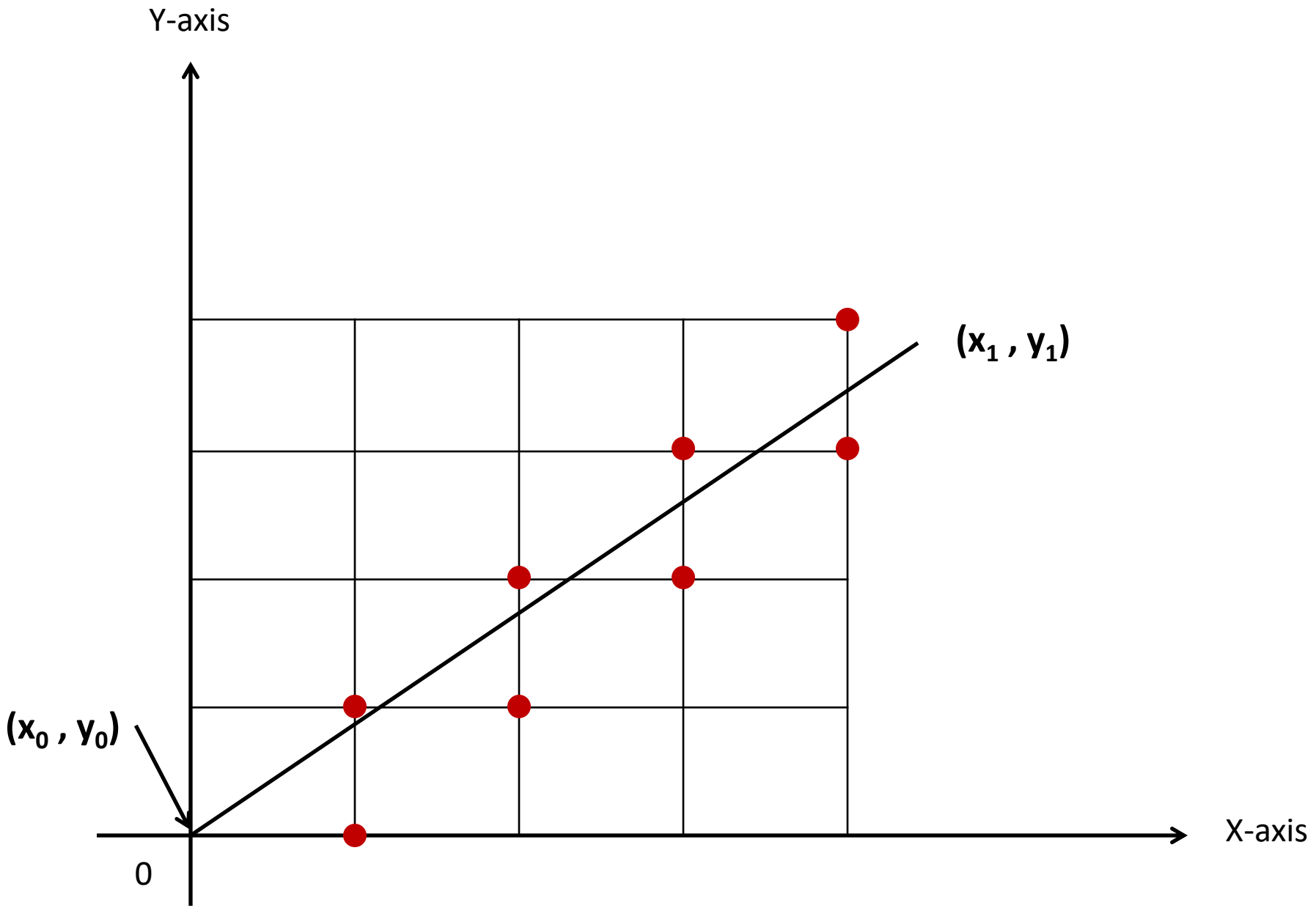
- **SLOPE INTERCEPT LINE EQUATION**

$$y = mx + b$$

- Where, **m = slope** and **b = y intercept**

- And $m = \frac{dy}{dx}$ **or** $\frac{\Delta y}{\Delta x} = \frac{y2-y1}{x2-x1}$

- And $b = y1 - mx1$


Y-axis
(x_1 , y_1)
(x_0 , y_0)
X-axis
0

LINEAR REGRESSION

- **Plot initial points** $x = x_0 , y = y_0$

- **CASE I** – if **m < 1** (**x is increasing more than y**)

- Then find new datapoints of (x, y)
- $x_{(new\ value)} = x_{(old\ value)} +$ **1 and**
- $y_{(new\ value)} = y_{(old\ value)} + m$

- Repeat the above equations till we get the following condition
- $x_0 = x_1$

LINEAR REGRESSION

- **Plot initial points** $x = x_0 , y = y_0$
- **CASE II** – if **m > 1** (**y is increasing more than x**)
- Then find new datapoints of (x, y)
- $x_{(new\ value)} = x_{(old\ value)} + \frac{1}{m}$
- $y_{(new\ value)} = y_{(old\ value)} + 1$
- Repeat the above equations till we get the following condition
- $y_0 = y_1$

LINEAR REGRESSION

- **LINEAR REGRESSION ALGORITHM STEPS**
- **STEP 1:** Identify the initial points with respect to **x** and **y** axis.
- **STEP 2:** Calculate the value of **slope(m).**
- **STEP 3:** Based of value of **slope(m)** identify the **case(I or II).**
- **STEP 4:** If **m<1,** ***x increment more than y*** and if **m>1**, ***y increments more than x***.
- **STEP 5:** Calculate the new datapoints of **x** and **y** according to the **case(I or II).**
- **STEP 6:** Repeat the algorithm until it reach the condition.

x_0 = x1 for m<1 or y_0 = y1 for m>1

LINEAR REGRESSION EXAMPLES

EXAMPLE 1

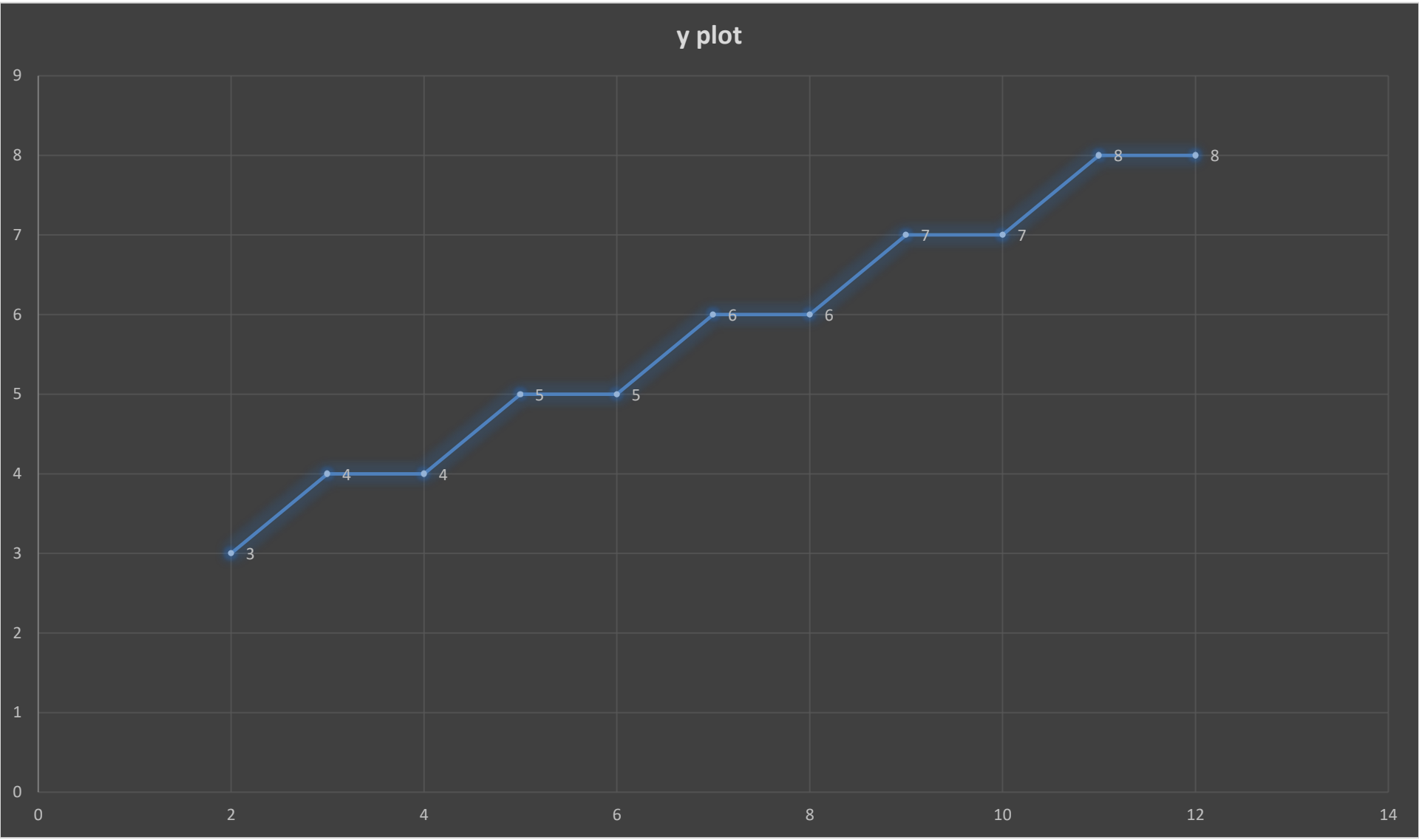
- **Input Coordinates A(2,3) & B(12,8)**
- ***x1 = 2, y1 = 3***
- ***x2 = 12, y2 = 8***
- Slope $m = \frac{\Delta y}{\Delta x} = \frac{y2-y1}{x2-x1} = \frac{8-3}{12-2} = \frac{5}{10} = \mathbf{0.5}$

- **m < 1** (**x increment more than y**)
- Find the new datapoints of (x, y)
- $\boldsymbol{x}_{(\boldsymbol{new\ values})} = \boldsymbol{x}_{(\boldsymbol{old\ values})} + \mathbf{1}$ and
- $\boldsymbol{y}_{(\boldsymbol{new\ values})} = \boldsymbol{y}_{(\boldsymbol{old\ values})} + \boldsymbol{m}$
- Repeat the above equations till we get the following condition
- $\boldsymbol{x_0 = x_1}$

EXAMPLE 1

Initial points: ***x1 = 2, y1 = 3*** ***Destination points:*** ***x2 = 12, y2 = 8***

$x = x_0 + 1$	x	$y = y_0 + m$	y	x-plotted	y-plotted
--	**2**	--	**3**	**2**	**3**
2 + 1	**3**	**3** + 0.5	**3.5**	**3**	**4**
3 + 1	**4**	**3.5** + 0.5	**4**	**4**	**4**
4 + 1	**5**	**4** + 0.5	**4.5**	**5**	**5**
5 + 1	**6**	**4.5** + 0.5	**5**	**6**	**5**
6 + 1	**7**	**5** + 0.5	**5.5**	**7**	**6**
7 + 1	**8**	**5.5** + 0.5	**6**	**8**	**6**
8 + 1	**9**	**6** + 0.5	**6.5**	**9**	**7**
9 + 1	**10**	**6.5** + 0.5	**7**	**10**	**7**
10 + 1	**11**	**7** + 0.5	**7.5**	**11**	**8**
11 + 1	**12**	**7.5** + 0.5	**8**	**12**	**8**


y plot
9
8
7
6
5
4
3
2
1
0
0
2
4
6
8
10
12
14
3
4
4
5
5
6
6
7
7
8
8

EXAMPLE 2

- **Input Coordinates A(0,0) & B(4,6)**
- ***x1 = 0, y1 = 0***
- ***x2 = 4, y2 = 6***
- Slope $m = \frac{\Delta y}{\Delta x} = \frac{y2-y1}{x2-x1} = \frac{6-0}{4-0} = \frac{6}{4} = \mathbf{1.5}$

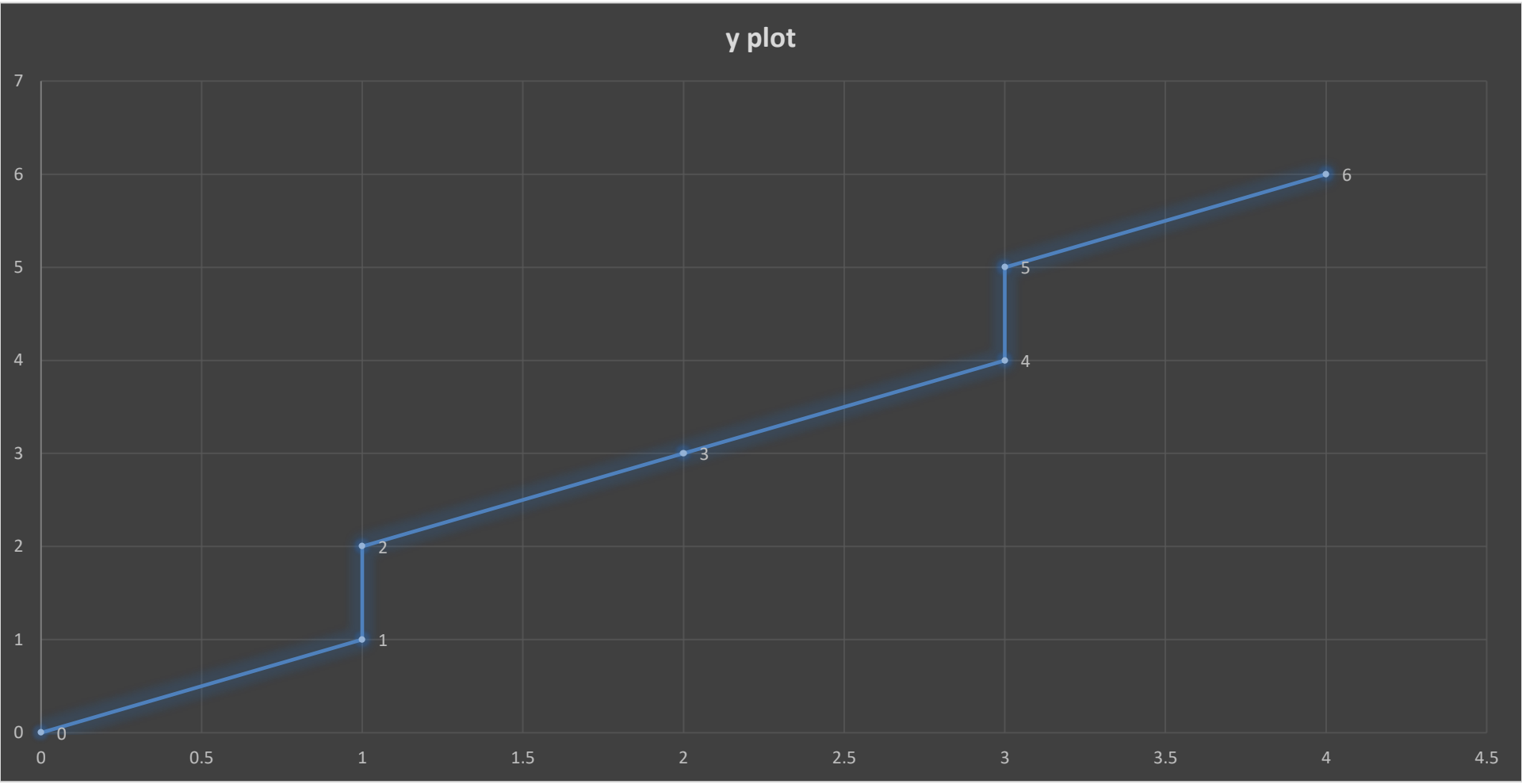
- **m > 1 (y increment more than x)**
- Find the new datapoints of (x, y)
- $\boldsymbol{x_{(new\ value)} = x_{(old\ value)} + \frac{1}{m}}$
- $\boldsymbol{y_{(new\ value)} = y_{(old\ value)} + 1}$

- Repeat the above equations till we get the following condition
- $\boldsymbol{y_0 = y_1}$

EXAMPLE 2

Initial points: ***x1 = 0, y1 = 0*** ***Destination points:*** ***x2 = 4, y2 = 6***

$x = x + \frac{1}{m}$	x	$y = y + 1$	y	x-plotted	y-plotted
--	**0**	--	**0**	**0**	**0**
0 + 0.66	**0.66**	**0** + 1	**1**	**1**	**1**
0.66 + 0.66	**1.32**	**1** + 1	**2**	**1**	**2**
1.32 + 0.66	**1.98**	**2** + 1	**3**	**2**	**3**
1.98 + 0.66	**2.64**	**3** + 1	**4**	**3**	**4**
2.64 + 0.66	**3.3**	**4** + 1	**5**	**3**	**5**
3.3 + 0.66	**3.96**	**5** + 1	**6**	**4**	**6**


y plot
7
6
5
4
3
2
1
0
0
0.5
1
1.5
2
2.5
3
3.5
4
4.5
0
1
2
3
4
5
6

LINEAR REGRESSION EXAMPLES TO BE SOLVED BY STUDENTS

LINEAR REGRESSION

- **EXAMPLE-1**
- Consider the Input Coordinates **A(4,5)** & **B(14,10).** Calculate the **slope(m)** and find new **x** and **y** datapoints.

- **EXAMPLE-2**
- Consider the Input Coordinates **A(0,0)** & **B(6,8).** Calculate the **slope(m)** and find new **x** and **y** datapoints.

EXAMPLE 1

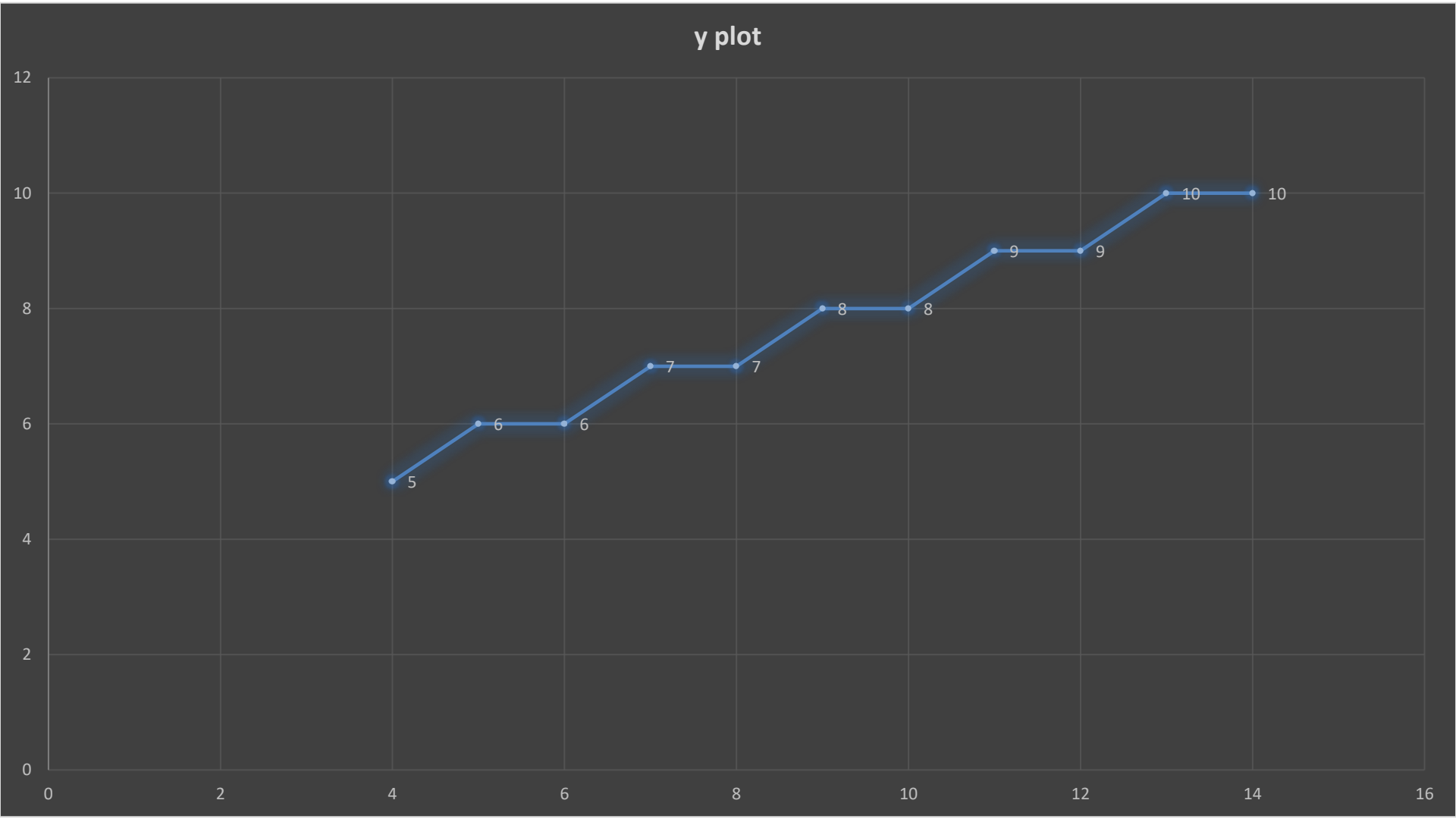
- **Input Coordinates A(4,5) & B(14,10)**
- ***x1 = 4, y1 = 5***
- ***x2 = 14, y2 = 10***
- Slope $m = \frac{\Delta y}{\Delta x} = \frac{y2-y1}{x2-x1} = \frac{10-5}{14-4} = \frac{5}{10} = \mathbf{0.5}$

- **m < 1** (**x increment more than y**)
- Find the new datapoints of (x, y)
- $\boldsymbol{x}_{(\boldsymbol{new\ values})} = \boldsymbol{x}_{(\boldsymbol{old\ values})} + \mathbf{1}$ and
- $\boldsymbol{y}_{(\boldsymbol{new\ values})} = \boldsymbol{y}_{(\boldsymbol{old\ values})} + \boldsymbol{m}$
- Repeat the above equations till we get the following condition
- $\boldsymbol{x_0 = x_1}$

EXAMPLE 1

Initial points: ***x1 = 4, y1 = 5*** ***Destination points:*** ***x2 = 14, y2 = 10***

$x = x_0 + 1$	x	$y = y_0 + m$	y	x-plotted	y-plotted
--	**4**	--	**5**	**4**	**5**
4 + 1	**5**	**5** + 0.5	**5.5**	**3**	**6**
3 + 1	**6**	**5.5** + 0.5	**6**	**4**	**6**
4 + 1	**7**	**6** + 0.5	**6.5**	**5**	**7**
5 + 1	**8**	**6.5** + 0.5	**7**	**6**	**7**
6 + 1	**9**	**7** + 0.5	**7.5**	**7**	**8**
7 + 1	**10**	**7.5** + 0.5	**8**	**8**	**8**
8 + 1	**11**	**8** + 0.5	**8.5**	**9**	**9**
9 + 1	**12**	**8.5** + 0.5	**9**	**10**	**9**
10 + 1	**13**	**9** + 0.5	**9.5**	**11**	**10**
11 + 1	**14**	**9.5** + 0.5	**10**	**14**	**10**


y plot
12
10
8
6
4
2
0
0
2
4
6
8
10
12
14
16
5
6
6
7
7
8
8
9
9
10
10

EXAMPLE 2

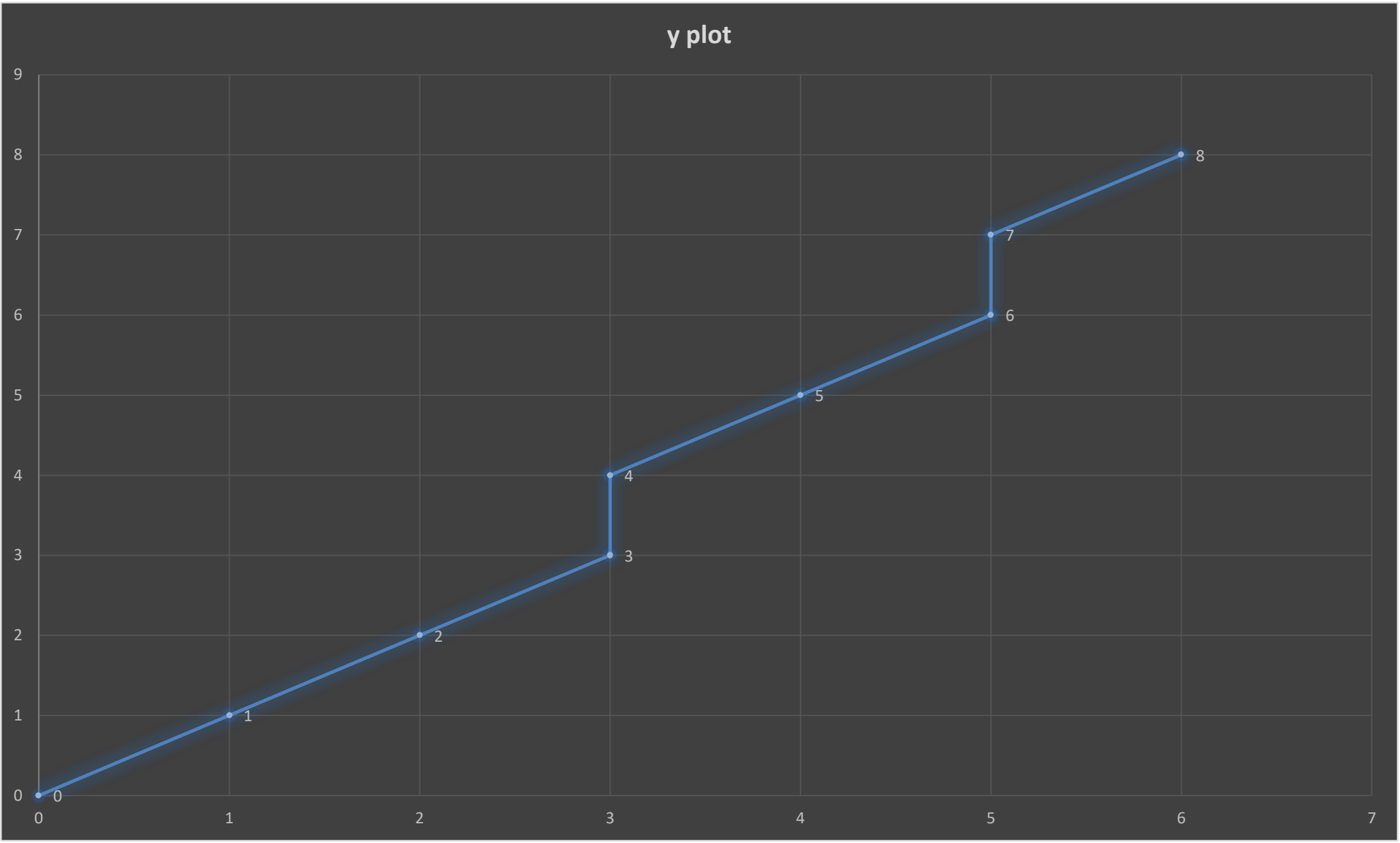
- **Input Coordinates A(0,0) & B(6,8)**
- ***x1 = 0, y1 = 0***
- ***x2 = 6, y2 = 8***
- Slope $m = \frac{\Delta y}{\Delta x} = \frac{y2-y1}{x2-x1} = \frac{8-0}{6-0} = \frac{8}{6} = \mathbf{1.3}$

- **m > 1** (**y increment more than x**)
- Find the new datapoints of (x, y)
- $\boldsymbol{x_{(new\ values)} = x_{(old\ values)} + 1/m}$
- $\boldsymbol{y_{(new\ values)} = y_{(old\ values)} + 1}$
- Repeat the above equations till we get the following condition
- $\boldsymbol{y_0 = y_1}$

EXAMPLE 2

Initial points: ***x1 = 0, y1 = 0*** ***Destination points:*** ***x2 = 6, y2 = 8***

$x = x + \frac{1}{m}$	x	$y = y + 1$	y	x-plotted	y-plotted
--	**0**	--	**0**	**0**	**0**
0 + 0.7	**0.7**	**0** + 1	**1**	**1**	**1**
0.7 + 0.7	**1.4**	**1** + 1	**2**	**2**	**2**
1.4 + 0.7	**2.1**	**2** + 1	**3**	**3**	**3**
2.1 + 0.7	**2.8**	**3** + 1	**4**	**3**	**4**
2.8 + 0.7	**3.5**	**4** + 1	**5**	**4**	**5**
3.5 + 0.7	**4.2**	**5** + 1	**6**	**5**	**6**
4.2 + 0.7	**4.9**	**6** + 1	**7**	**5**	**7**
5.6 + 0.7	**5.6**	**7** + 1	**8**	**6**	**8**


y plot
9
8
7
6
5
4
3
2
1
0
0
1
2
3
4
5
6
7
0
1
2
3
4
5
6
7
8

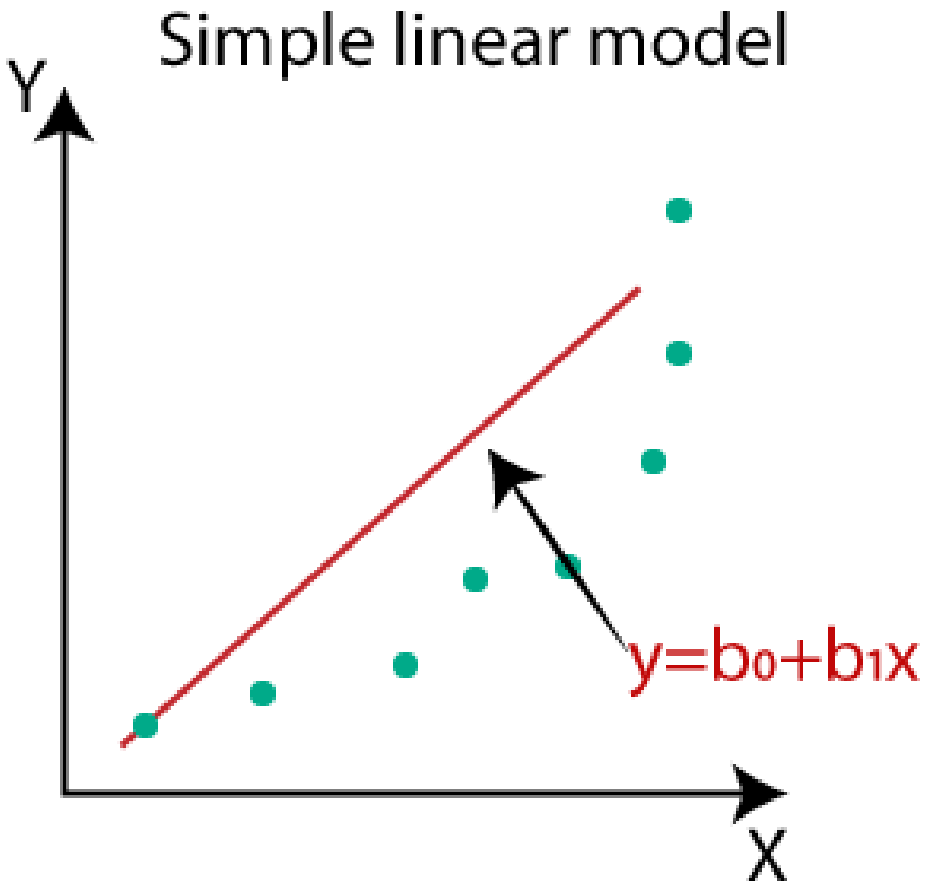
POLYNOMIAL REGRESSION ALGORITHM

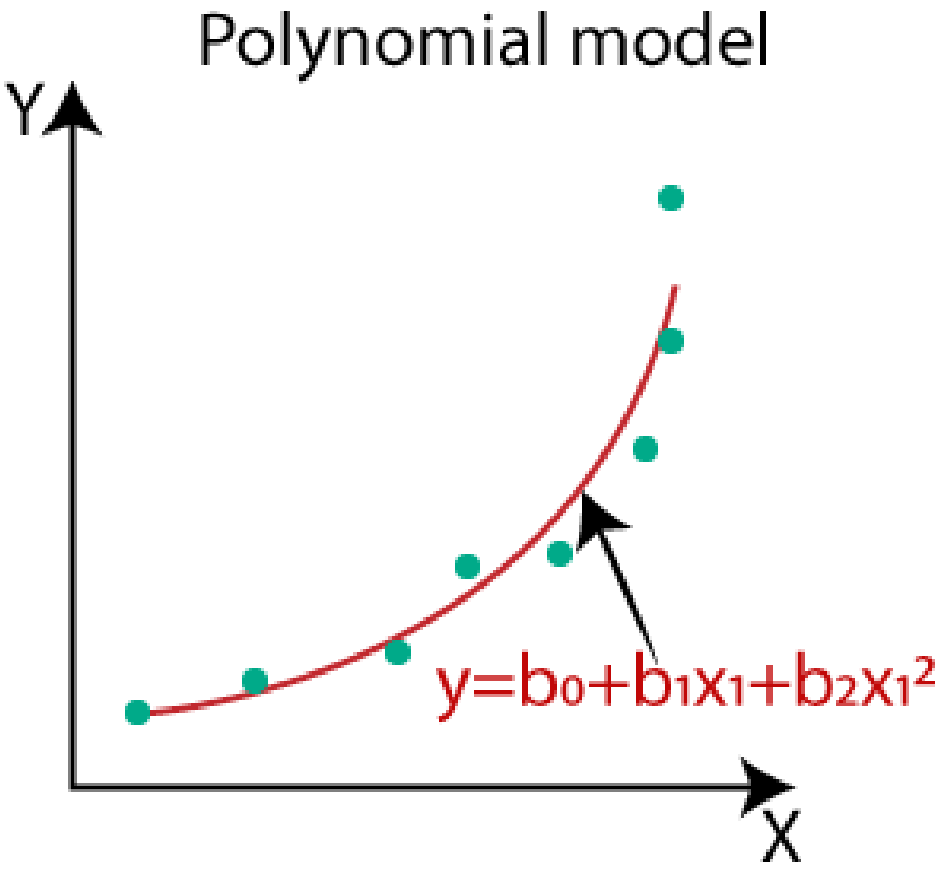
POLYNOMIAL REGRESSION

- **INTRODUCTION**
- **Polynomial regression** is a kind of **linear regression** in which the relationship shared between the **dependent (y)** and **independent (x) variables** is modeled as the ***nth degree*** of the **polynomial**.

POLYNOMIAL REGRESSION

- **INTRODUCTION**
- The algorithm of **linear regression** works only when the **regression** in the **data** is **linear**.
- **Polynomial regression** can be considered one of the exceptional cases of **multiple linear regression models**.
- In other words, it is a **linear regression** type containing **dependent (y)** and **independent variables (x)**, and they both share a ***curvilinear relationship***.
- A polynomial relationship is fitted in the data.


Simple linear model
Y
X
y=b0+b1x


Polynomial model
Y
X
y=b0+b1x1+b2x1²

POLYNOMIAL REGRESSION

- **TYPES OF POLYNOMIAL REGRESSION**
- Since there is no limit to the degree in a polynomial equation, and it can go up to the **nth value**, numerous kinds of polynomial regression exist.
- **Linear, when the degree is 1 (x^1).**
- **Quadratic, the degree of this equation is 2 (x^2).**
- **Cubic with a degree as three continues, based on the degree used (x^3).**

Polynomials	Form	Degree	Examples
Linear Polynomial	$p(x): ax+b, a \neq 0$	Polynomial with Degree 1	$x + 8$
Quadratic Polynomial	$p(x): ax^2+b+c, a \neq 0$	Polynomial with Degree 2	$3x^2-4x+7$
Cubic Polynomial	$p(x): ax^3+bx^2+cx, a \neq 0$	Polynomial with Degree 3	$2x^3+3x^2+4x+6$

REGULARIZATION TECHNIQUES

REGULARIZATION TECHNIQUES

- ***Regularization is a technique used to prevent overfitting by adding a penalty term to the model's objective function during training.***

REGULARIZATION TECHNIQUES

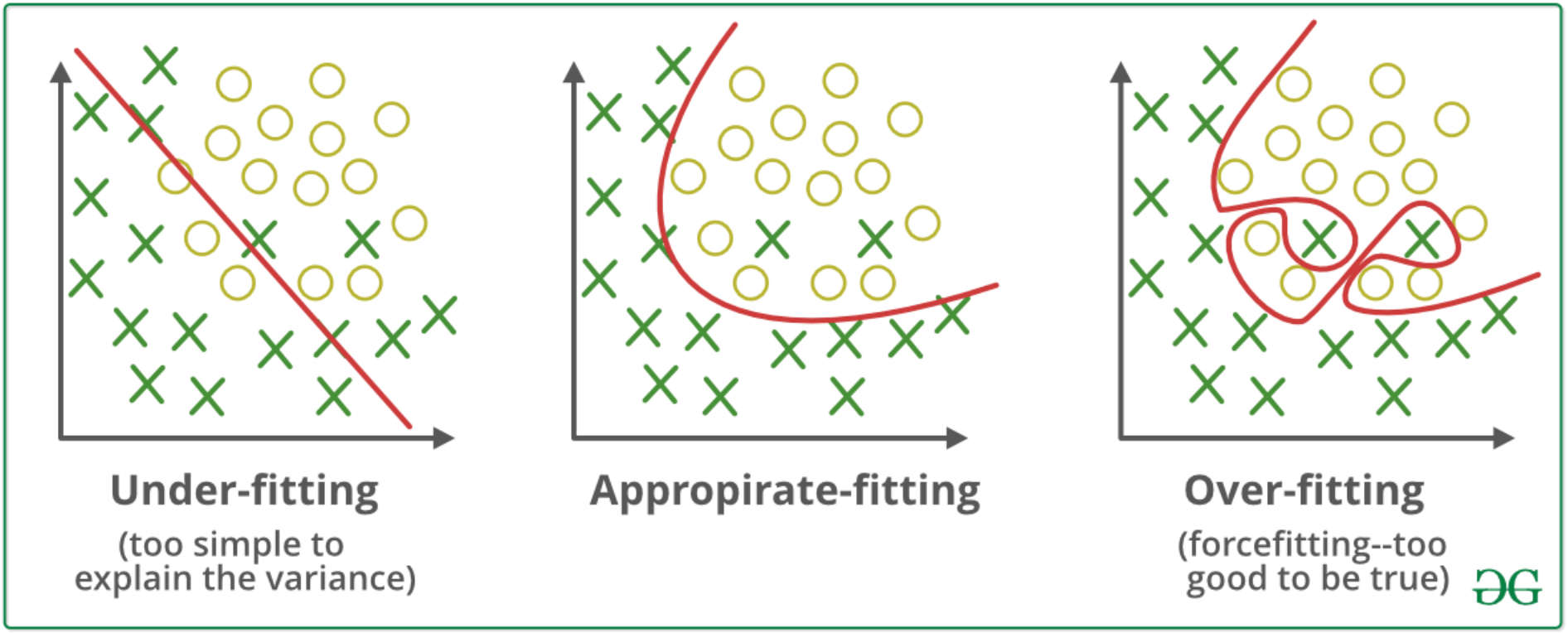
- **OVERFITTING**
- ***Overfitting happens when a model learns too much from the training data, including details that don't matter (like noise or outliers).***

- Overfitting models are like **students who memorize answers instead of understanding the topic**. They do well in ***practice tests (training)*** but struggle in ***real exams (testing).***

REGULARIZATION TECHNIQUES

- **UNDERFITTING**
- ***Underfitting is the opposite of overfitting. It happens when a model is too simple to capture what's going on in the data.***

- Underfitting models are like **students who don't study enough**. They don't do well in ***practice tests*** or ***real exams***.


Under-fitting
(too simple to explain the variance)
Appropirate-fitting
Over-fitting
(forcefitting--too good to be true)

REGULARIZATION TECHNIQUES

- **TYPES OF REGULARIZATION**
- L1 Regularization (Lasso)
- L2 Regularization (Ridge)

REGULARIZATION TECHNIQUES

- **LASSO REGULARIZATION**
- A regression model which uses the L1 Regularization technique is called **LASSO** (**Least Absolute Shrinkage and Selection Operator**) regression.

- Lasso Regression adds the "**absolute value of magnitude**" of the coefficient as a **penalty** term to the **loss function(L).**

REGULARIZATION TECHNIQUES

- **RIDGE REGULARIZATION**
- A regression model that uses the **L2 regularization** technique is called **Ridge regression**.

- **Ridge regression** adds the "**squared magnitude**" of the coefficient as a **penalty** term to the **loss function(L).**

CLASSIFICATION

CLASSIFICATION

- **CLASSIFICATION**
- ***Classification models are used to generate conclusions from observed values in the categorical form.***

LOGISTIC REGRESSION ALGORITHM

LOGISTIC REGRESSION

- **INTRODUCTION**
- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique.

- ***It is used for predicting the categorical dependent variable using a given set of independent variables.***

LOGISTIC REGRESSION

- **INTRODUCTION**
- **Logistic regression** predicts the output of a categorical dependent variable.
- Therefore, the outcome must be a **categorical** or **discrete** value.
- It can be either **Yes** or **No**, **0** or **1**, **true** or **False**, etc. but instead of giving the exact value as 0 and 1, it gives the probabilistic values which lie between 0 and 1.

LOGISTIC REGRESSION

- **INTRODUCTION**
- In **Logistic regression**, instead of fitting a regression line, we fit an "**S**" shaped **logistic function**, which predicts two maximum values (**0 or 1**)

- The curve from the **logistic function** indicates the likelihood of something such as whether the cells are cancerous or not, a mouse is obese or not based on its weight, etc.

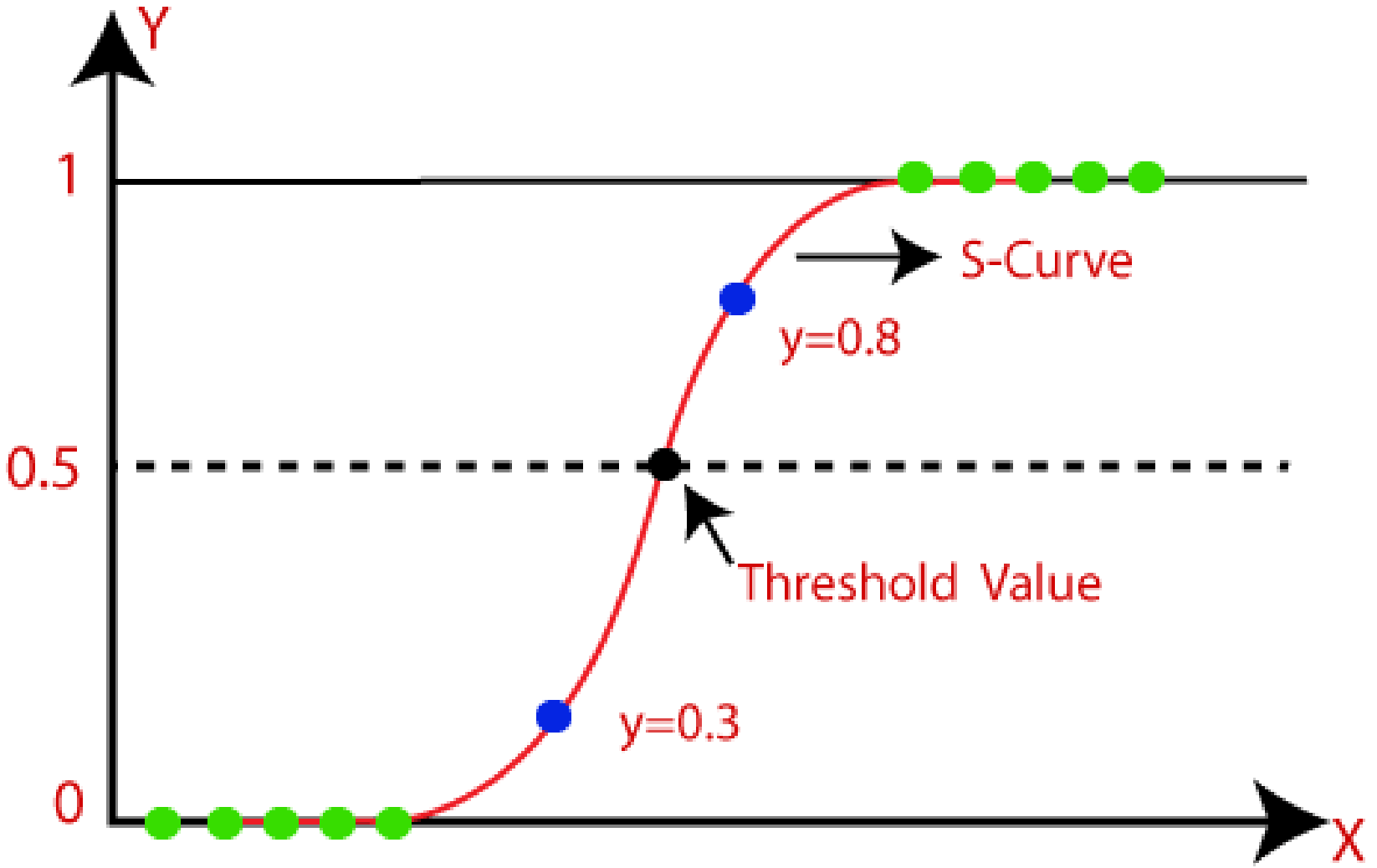
LOGISTIC REGRESSION

- **INTRODUCTION**

$$S(x) = \frac{1}{1 + e^{-x}}$$

- Where,
- e = Euler's number (2.71828)

LOGISTIC REGRESSION



LOGISTIC REGRESSION

- **Type of Logistic Regression:**
- On the basis of the categories, Logistic Regression can be classified into three types:

- **Binomial:** In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.

LOGISTIC REGRESSION

- **Type of Logistic Regression:**

- **Multinomial:** In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as "cat", "dogs", or "sheep"

LOGISTIC REGRESSION

- **Type of Logistic Regression:**

- **Ordinal:** In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as "low", "Medium", or "High".

LOGISTIC REGRESSION ALGORITHM EXAMPLE

LOGISTIC REGRESSION

- **Example: Consider a dataset of pass or fail of 5 students. Use logistic regression as classifier to answer following questions.**
- 1. Calculate the **probability** of **pass** for the student who studied for **33 hours**.
- 2. At least how many **hours** student should study that makes he will **pass** the course with the **probability** of more than **95%.**

LOGISTIC REGRESSION

- **Example:**

HOURS	PASS (1) / FAIL (0)
29	0
15	0
33	1
28	1
39	1

LOGISTIC REGRESSION

- **Solution**: Assume the model of optimizer for odds of passing
- $\boldsymbol{log\ (odds) = -64 + 2\ * hours}$
- 1. Calculate the **probability** of **pass** for the student who studied for **33 hours**.

$$S(x) = \frac{1}{1 + e^{-x}}$$

- $\boldsymbol{x = -64 + 2\ * 33 = -64 + 66 = 2}$
- $\boldsymbol{p = \frac{1}{1+e^{-x}} = \frac{1}{1+0.1353} = \frac{1}{1.1353} = 0.88}$
- If a student studies for **33 hours** then there is probability of **88%** chance that student will **pass**.

LOGISTIC REGRESSION

- 2. At least how many **hours** student should study that makes he will **pass** the course with the **probability** of more than **95%.**

- $\boldsymbol{p = \frac{1}{1+e^{-x}} = 0.95}$
- $0.95 * (1 + e^{-x}) = 1$
- $0.95 * e^{-x} = 1 - 0.95$
- $e^{-x} = 0.\frac{005}{0.95} = \mathbf{0.0526}$
- $\ln(\boldsymbol{e^{-x}}) = \ln(0.0526)$
- $-x = \ln(0.0526) = -2.94$
- $x = \mathbf{2.94}$

LOGISTIC REGRESSION

- $x = \mathbf{2.94}$
- $x = -64 + 2 \ * hours$
- $2.94 = -64 + 2 * hours$
- $2 * hours = 2.94 + 64 = 66.94$
- $hours = 66.94/2$
- $hours = \mathbf{33.47}$
- The student should study at least **33.47 hours** , so that he will pass the exam with more than **95% probability**.

LOGISTIC REGRESSION ALGORITHM EXAMPLE TO BE SOLVED BY STUDENTS

LOGISTIC REGRESSION

- **Example: Consider a dataset of good or bad of 5 products. Use logistic regression as classifier to answer following questions.**
- 1. Calculate the **probability** of **good** for the product whose rating is more than **4**.
- 2. Calculate the **probability** of **bad** for the product whose rating is less than **2**
- 3. What should be the rating of a product that will with the **probability** of more than **3.5.**

LOGISTIC REGRESSION

- **Example:**

RATING	GOOD (1) / BAD (0)
1	0
2	0
3	1
4	1
4.5	1

KNN ALGORITHM

K-NEAREST NEIGHBOUR (KNN)

- **INTRODUCTION**
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity.
- This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.

K-NEAREST NEIGHBOUR (KNN)

- **INTRODUCTION**
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.

- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.

K-NEAREST NEIGHBOUR (KNN)

- **EXAMPLE**
- Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure.
- Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.

K-NEAREST NEIGHBOUR (KNN)

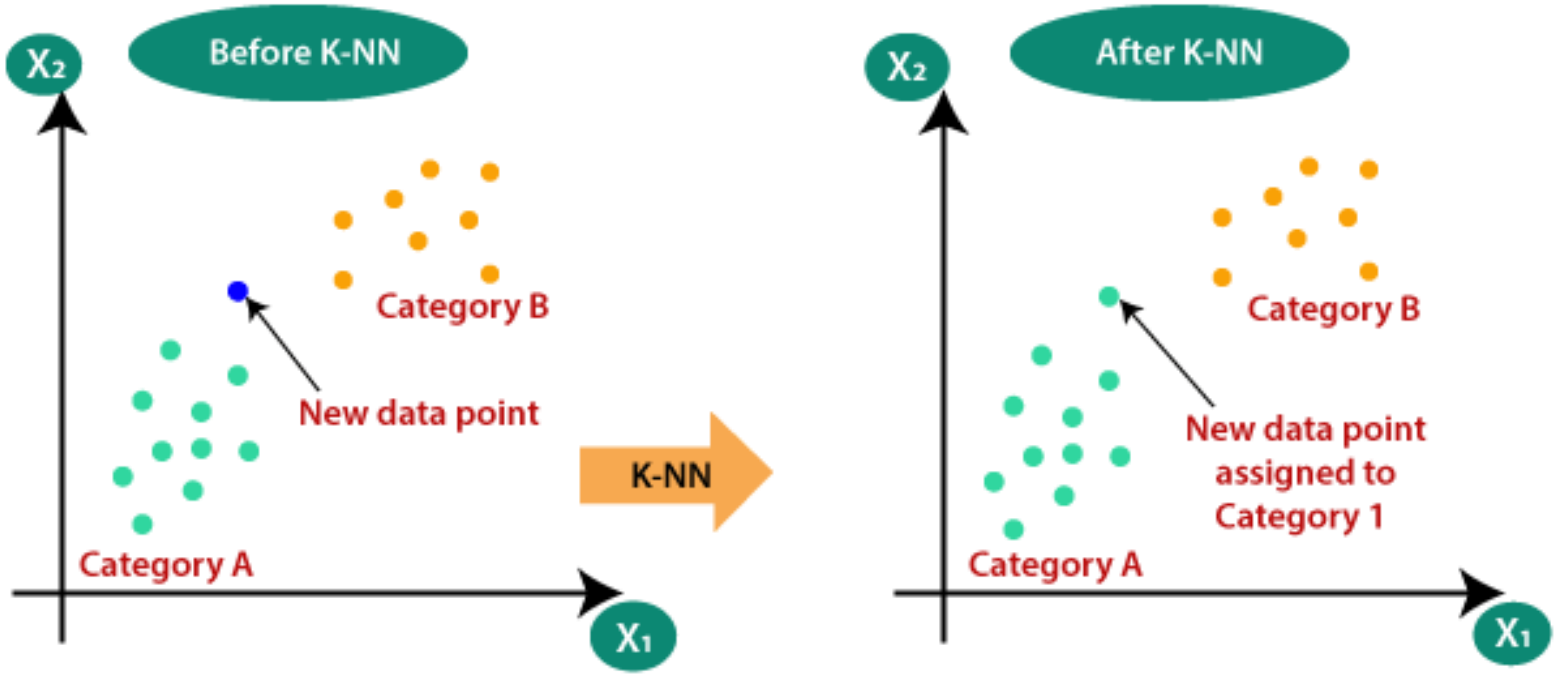
K-NEAREST NEIGHBOUR (KNN)

- **NEED OF KNN ALGORITHM**
- Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x1, so this data point will lie in which of these categories.

- To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset.

K-NEAREST NEIGHBOUR (KNN)

- **NEED OF KNN ALGORITHM**
- Consider the below diagram:

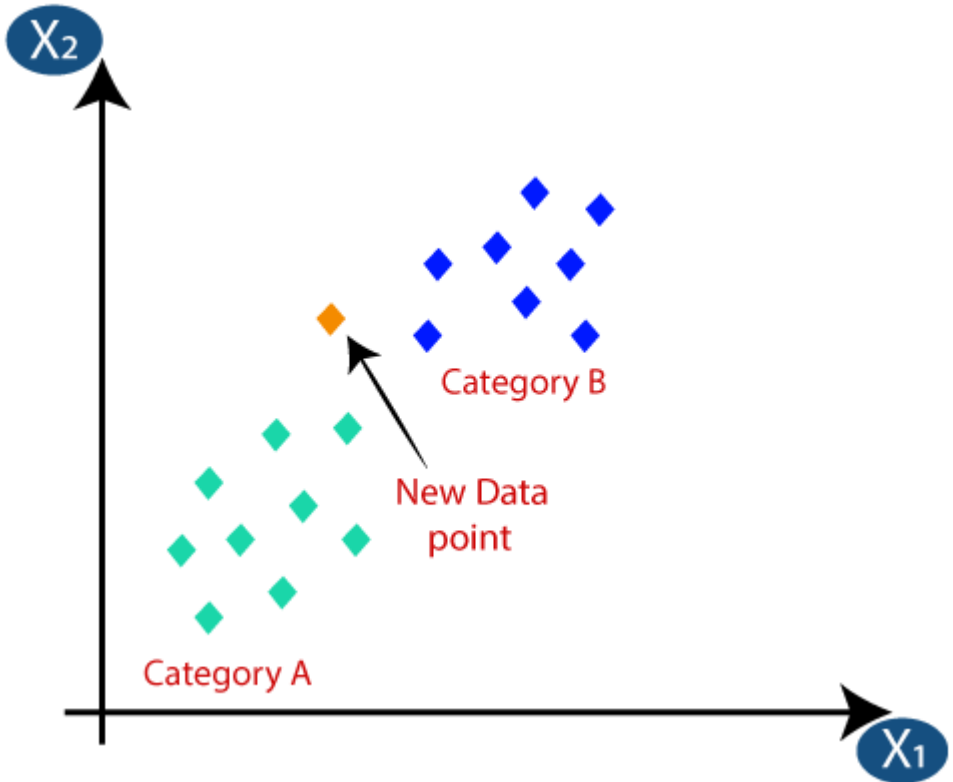


K-NEAREST NEIGHBOUR (KNN)

- **WORKING OF KNN ALGORITHM**
- The K-NN working can be explained on the basis of the below algorithm:
- **Step-1:** Select the number K of the neighbors
- **Step-2:** Calculate the Euclidean distance of K number of neighbors
- **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- **Step-4:** Among these k neighbors, count the number of the data points in each category.
- **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.
- **Step-6:** Our model is ready.

K-NEAREST NEIGHBOUR (KNN)

- **WORKING OF KNN ALGORITHM**
- Suppose we have a new data point and we need to put it in the required category. Consider the below image:



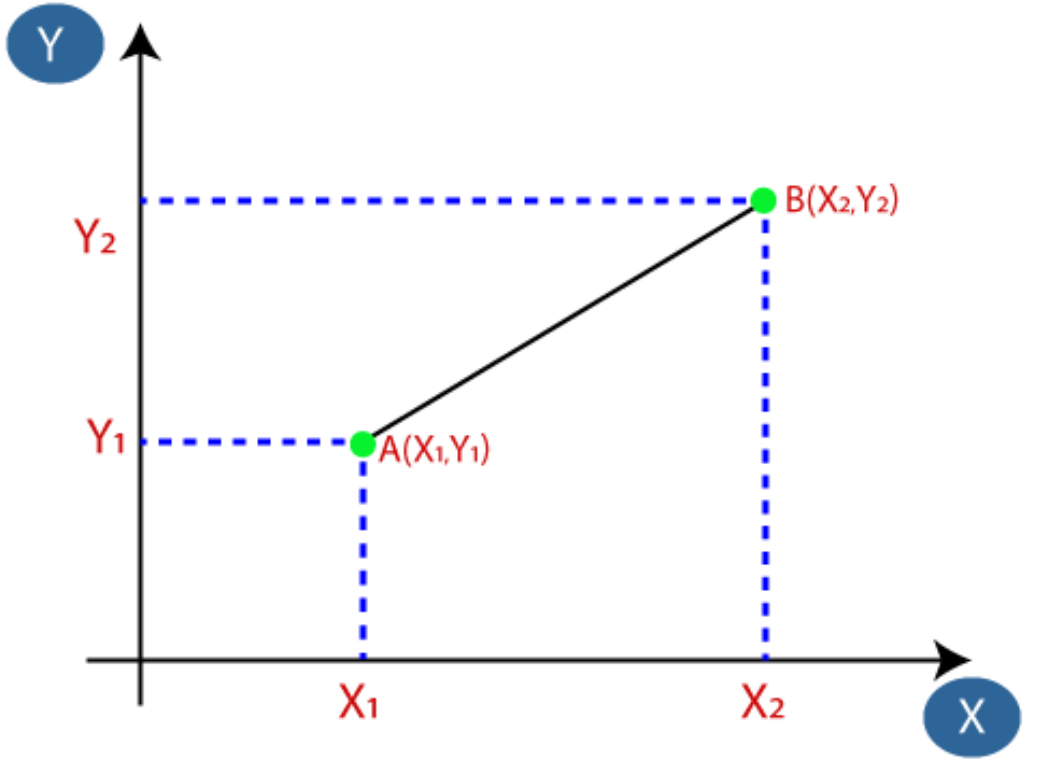
K-NEAREST NEIGHBOUR (KNN)

- **WORKING OF KNN ALGORITHM**
- Firstly, we will choose the number of neighbors, so we will choose the k=5.

- Next, we will calculate the Euclidean distance between the data points. The Euclidean distance is the distance between two points, which we have already studied in geometry.

K-NEAREST NEIGHBOUR (KNN)

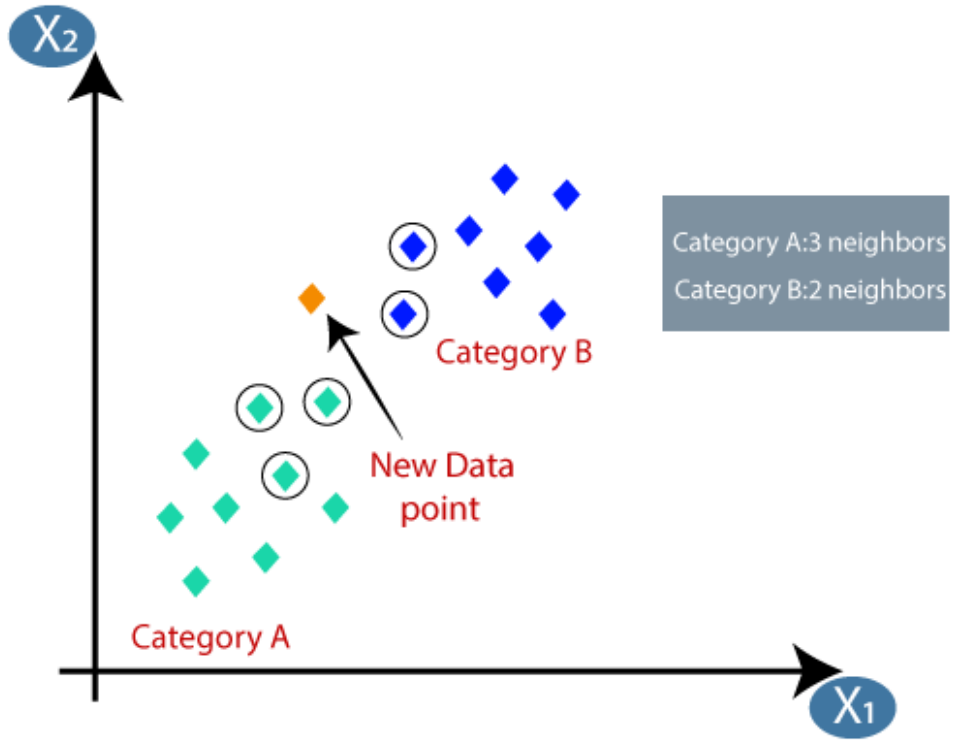
- **WORKING OF KNN ALGORITHM**
- It can be calculated as:



Euclidean Distance between A_1 and $B_2 = \sqrt{(X_2-X_1)^2+(Y_2-Y_1)^2}$

K-NEAREST NEIGHBOUR (KNN)

- **WORKING OF KNN ALGORITHM**
- By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B. Consider the below image:



K-NEAREST NEIGHBOUR (KNN)

- **ADVANTAGES OF KNN ALGORITHM**
  - It is simple to implement.
  - It is robust to the noisy training data
  - It can be more effective if the training data is large.

- **DISADVANTAGES OF KNN ALGORITHM**
  - Always needs to determine the value of K which may be complex some time.
  - The computation cost is high because of calculating the distance between the data points for all the training samples.

KNN ALGORITHM EXAMPLE

K-NEAREST NEIGHBOUR (KNN)

Sepal Length	Sepal Width	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3.0	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Virginica
6.1	2.8	Verscicolor
7.3	2.9	Virginica
6.0	2.7	Verscicolor
5.8	2.8	Virginica
6.3	2.3	Verscicolor
5.1	2.5	Verscicolor
6.3	2.5	Verscicolor
5.5	2.4	Verscicolor

EXAMPLE: Consider the dataset of different species of flowers and calculate the nearest distance using Euclidean formula with the new instance.

Finally consider the values of **K = 1, 2 and 5** and find the calculate the nearest distance.

NEW INSTANCE	SEPAL LENGTH (x)	SEPAL WIDTH (y)	SEPAL SPECIES
	5.2	3.1	?

K-NEAREST NEIGHBOUR (KNN)

Step 1: Find Distance

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(x-a)^2 + (y-b)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(5.2-5.3)^2 + (3.1-3.7)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = 0.608$$

Sepal Length (a)	Sepal Width (b)	Species	Distance
5.3	3.7	Setosa	0.608

Euclidean Distance calculated with NEW INSTANCE

Sepal Length	Sepal Width	Species	Distance
5.3	3.7	Setosa	0.608
5.1	3.8	Setosa	0.707
7.2	3.0	Virginica	2.002
5.4	3.4	Setosa	0.36
5.1	3.3	Setosa	0.22
5.4	3.9	Setosa	0.82
7.4	2.8	Virginica	2.22
6.1	2.8	Verscicolor	0.94
7.3	2.9	Virginica	2.1
6.0	2.7	Verscicolor	0.89
5.8	2.8	Virginica	0.67
6.3	2.3	Verscicolor	1.36
5.1	2.5	Verscicolor	0.60
6.3	2.5	Verscicolor	1.25
5.5	2.4	Verscicolor	0.75

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Verscicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Verscicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Verscicolor	1.36	12
5.1	2.5	Verscicolor	0.60	4
6.3	2.5	Verscicolor	1.25	11
5.5	2.4	Verscicolor	0.75	7

Step 2: Find Rank

Find the rank based on *minimum* distance to *maximum* distance

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Verscicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Verscicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Verscicolor	1.36	12
5.1	2.5	Verscicolor	0.60	4
6.3	2.5	Verscicolor	1.25	11
5.5	2.4	Verscicolor	0.75	7

Step 3: Find the Nearest Neighbor

If k = 1 – Setosa

If k = 2 – Setosa

If k = 5 – Setosa

As

K = 1 – Setosa

K = 2 – Setosa

K = 3 – Setosa

K = 4 – Verscicolor

K = 5 - Virginica

For K = 5, the maximum count is of SETOSA. So K = 5 belongs to SETOSA

K-NEAREST NEIGHBOUR (KNN)

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Verscicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Verscicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Verscicolor	1.36	12
5.1	2.5	Verscicolor	0.60	4
6.3	2.5	Verscicolor	1.25	11
5.5	2.4	Verscicolor	0.75	7

RANK (K)	SEPAL SPECIES for NEW INSTANCE
1	Setosa
2	Setosa
3	Setosa
4	Setosa
5	Setosa
6	Setosa
7	Setosa
8	Setosa
9	Setosa
10	Setosa
11	Setosa / Verscicolor
12	Verscicolor
13	Verscicolor
14	Verscicolor
15	Verscicolor

NEW INSTANCE	SEPAL LENGTH (x)	SEPAL WIDTH (y)	SEPAL SPECIES
	5.2	3.1	?

KNN ALGORITHM EXAMPLE TO BE SOLVED BY STUDENTS

K-NEAREST NEIGHBOUR (KNN)

Height (CM)	Weight (KG)	Class
167	51	Underweight
182	62	Normal
176	69	Normal
173	64	Normal
172	65	Normal
174	56	Underweight
169	58	Normal
173	57	Normal
170	55	Normal

EXAMPLE: Consider the dataset of **Height (cm)** and **Weight (kg).** Calculate the nearest distance using Euclidean formula with the new instance.

Calculate the nearest distance for all k values (**1 to 9**).

Euclidean Distance Formula

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

NEW INSTANCE	Height (x2)	Weight (y2)	Class
	170	57	?

K-NEAREST NEIGHBOUR (KNN)

Height (CM)	Weight (KG)	Class	Distance
167	51	Underweight	6.7
182	62	Normal	13
176	69	Normal	13.4
173	64	Normal	7.6
172	65	Normal	8.2
174	56	Underweight	4.1
169	58	Normal	1.4
173	57	Normal	3
170	55	Normal	2

Euclidean Distance Formula

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

NEW INSTANCE	Height (x)	Weight (y)	Class
	170	57	?

K-NEAREST NEIGHBOUR (KNN)

Height (CM)	Weight (KG)	Class	Distance	Rank
169	58	Normal	1.4	1
170	55	Normal	2	2
173	57	Normal	3	3
174	56	Underweight	4.1	4
167	51	Underweight	6.7	5
173	64	Normal	7.6	6
172	65	Normal	8.2	7
182	62	Normal	13	8
176	69	Normal	13.4	9

NEW INSTANCE	Height (x2)	Weight (y2)	Class
	170	57	?

K-NEAREST NEIGHBOUR (KNN)

Height (CM)	Weight (KG)	Class	Distance	Rank
169	58	Normal	1.4	1
170	55	Normal	2	2
173	57	Normal	3	3
174	56	Underweight	4.1	4
167	51	Underweight	6.7	5
173	64	Normal	7.6	6
172	65	Normal	8.2	7
182	62	Normal	13	8
176	69	Normal	13.4	9

RANK (K)	CLASS for NEW INSTANCE
1	Normal
2	Normal
3	Normal
4	Normal
5	Normal
6	Normal
7	Normal
8	Normal
9	Normal

NEW INSTANCE	Height (x2)	Weight (y2)	Class
	170	57	?

BAYES LEARNING

BAYESIAN (BAYES) LEARNING

- ***Bayes theorem helps to determine the probability of an event with random knowledge.***

BAYESIAN (BAYES) LEARNING

- The theorem can be mathematically expressed as:

- $P(A|B) = [P(B|A) . P(A)]/P(B)$

- where
  - **P(A|B)** is the **posterior** probability of **event A** given **event B**.
  - **(B|A)** is the **likelihood** of **event B** given **event A**.
  - **P(A)** is the **prior** probability of **event A**.
  - **P(B)** is the **total** probability of **event B**.

NAVIE BAYES (NB) ALGORITHM

NAVIE BAYES CLASSIFIER ALGORITM

- **Naïve Bayes algorithm** is a **supervised learning algorithm**, which is based on **Bayes theorem** and used for **solving classification problems**.
- It is mainly used in **text classification** that includes a **high-dimensional training dataset**.
- **Naïve Bayes Classifier** is one of the **simple** and most effective **classification algorithms** which helps in **building** the fast machine learning models that can make **quick predictions**.
- It is a **probabilistic classifier**, which means it **predicts** on the basis of the **probability** of an **object**.
- Some popular examples of Naïve Bayes Algorithm are **spam filtration, Sentimental analysis**, and **classifying articles**.

NAVIE BAYES CLASSIFIER ALGORITHM

- **Why is it called Naïve Bayes?**
- The Naïve Bayes algorithm is comprised of two words **Naïve** and **Bayes**, which can be described as:
- **Naïve:** It is called Naïve because it assumes that the **occurrence** of a certain **feature** is **independent** of the **occurrence** of **other features**. Such as if the fruit is identified on the bases of color, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple. Hence each feature individually contributes to identify that it is an apple without depending on each other.
- **Bayes:** It is called **Bayes** because it depends on the **principle** of **Bayes' Theorem**.

NAVIE BAYES CLASSIFIER ALGORITHM

- The formula for Bayes' theorem is given as:

- $P(A|B) = [P(B|A) . P(A)]/P(B)$

- Where,
  - **P(A|B)** is the **posterior** probability of **event A** given **event B**.
  - **(B|A)** is the **likelihood** of **event B** given **event A**.
  - **P(A)** is the **prior** probability of **event A**.
  - **P(B)** is the **total** probability of **event B**.

NAVIE BAYES CLASSIFIER ALGORITHM

- **WORKING OF NAVIE BAYES ALGORITHM**
- Suppose we have a **dataset** of **weather conditions** and corresponding target variable "**Play**". So using this dataset we need to decide that whether **we should play or not** on a **particular day** according to the **weather conditions**.

- So to solve this problem, we need to follow the below steps:
  - Convert the given dataset into **frequency tables**.
  - Generate **Likelihood table** by finding the **probabilities** of given **features**.
  - Now, use **Bayes theorem** to calculate the **posterior probability**.

NAVIE BAYES CLASSIFIER ALGORITM

- **ADVANTAGES OF NAVIE BAYES ALGORITHM**
- Naïve Bayes is one of the fast and easy ML algorithms to predict a class of datasets.
- It can be used for Binary as well as Multi-class Classifications.
- It performs well in Multi-class predictions as compared to the other Algorithms.
- It is the most popular choice for text classification problems.

- **DISADVANTAGES OF NAVIE BAYES ALGORITHM**
- Naive Bayes assumes that all features are independent or unrelated, so it cannot learn the relationship between features.

NB ALGORITHM EXAMPLE

NAVIE BAYES CLASSIFIER ALGORITHM

- **NAVIE BAYES ALGORITHM : EXAMPLE**
- **Problem: If the weather is *sunny*, then the Player should *play* or *not*?**

- **Solution: To solve this, first consider the below dataset:**

NAVIE BAYES CLASSIFIER ALGORITM

	Outlook	Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

NAVIE BAYES CLASSIFIER ALGORITHM

Frequency Table for weather condition

Weather	YES	NO
Overcast	5	0
Rainy	2	2
Sunny	3	2
Total	**10**	**4**

NAVIE BAYES CLASSIFIER ALGORITHM

Likelihood Table for weather condition

Weather	YES	NO	Total
Overcast	5	0	**5/14 = 0.35**
Rainy	2	2	**4/14 = 0.29**
Sunny	3	2	**5/14 = 0.35**
Total	**10/14 = 0.71**	**4/14 = 0.29**	

NAVIE BAYES CLASSIFIER ALGORITHM

- **NAVIE BAYES ALGORITHM : EXAMPLE**
- **Problem: If the weather is *sunny*, then the Player should *play* or *not*?**
- Applying Bayes' Theorem: $P(A|B)=[P(B|A) \ . P(A)]/P(B)$
- **P(Yes|Sunny)** = **[P(Sunny|Yes) * P(Yes)] / P(Sunny)**
  - **P(Sunny|Yes)** = 3/10 = **0.3**
  - **P(Sunny)** = **0.35**
  - **P(Yes)** = **0.71**

Weather	YES	NO	Total
Overcast	5	0	5/14 = 0.35
Rainy	2	2	4/14 = 0.29
Sunny	3	2	5/14 = 0.35
Total	10/14 = 0.71	4/14 = 0.29	

- So, **P(Yes|Sunny) = [0.3 * 0.71] / 0.35 = 0.60**

NAVIE BAYES CLASSIFIER ALGORITHM

- **NAVIE BAYES ALGORITHM : EXAMPLE**
- **Problem: If the weather is sunny, then the Player should play or not?**
- **P(No|Sunny) = [P(Sunny|No)*P(No)] / P(Sunny)**
  - **P(Sunny|NO)** = 2/4 = **0.5**
  - **P(No)** = **0.29**
  - **P(Sunny)** = **0.35**
- So, **P(No|Sunny) = [0.5*0.29]/0.35 = 0.41**
- So as we can see from the above calculation that **P(Yes|Sunny) > P(No|Sunny).** Hence on a **Sunny day**, Player **can play the game**.

Weather	YES	NO	Total
Overcast	5	0	5/14 = 0.35
Rainy	2	2	4/14 = 0.29
Sunny	3	2	5/14 = 0.35
Total	10/14 = 0.71	4/14 = 0.29	

NB ALGORITHM EXAMPLE TO BE SOLVED BY STUDENTS

NAVIE BAYES CLASSIFIER ALGORITHM

- **NAVIE BAYES ALGORITHM : EXAMPLE TO SOLVE**
- **Problem: Identify the probability of the FRUIT which will be choose based on the given attributes**
- **Consider the below dataset:**

FRUIT	YELLOW	SWEET	LONG	TOTAL AVAILABLE FRUIT
ORANGE	350	450	0	650
BANANA	400	300	350	400
OTHERS	50	100	50	150
TOTAL	800	850	400	1200

NAVIE BAYES CLASSIFIER ALGORITHM

- **NAVIE BAYES ALGORITHM : EXAMPLE-2**
- Applying Bayes‘ Theorem:
- $P(A|B)=[P(B|A) \ . P(A)]/P(B)$
- **P(Yellow | Orange) = [P(Orange | Yellow) * P(Yellow)] / P(Orange)**
- $[(\frac{350}{800}) * (\frac{800}{1200})]/(\frac{650}{1200}) = 0.53$
- So, **P(Yellow | Orange) = 0.53**

FRUIT	YELLOW	SWEET	LONG	TOTAL AVAILABLE FRUIT
ORANGE	350	450	0	650
BANANA	400	300	350	400
OTHERS	50	100	50	150
TOTAL	800	850	400	1200

NAVIE BAYES CLASSIFIER ALGORITHM

- Applying Bayes' Theorem:
- $P(A|B)=[P(B|A) \ . P(A)]/P(B)$
- **P(Sweet | Orange) = [P(Orange | Sweet) * P(Sweet)] / P(Orange)**
- $[\left(\frac{450}{850}\right) * (\frac{850}{1200})]/(\frac{650}{1200}) = 0.69$
- So, **P(Sweet | Orange) = 0.69**

FRUIT	YELLOW	SWEET	LONG	TOTAL AVAILABLE FRUIT
ORANGE	350	450	0	650
BANANA	400	300	350	400
OTHERS	50	100	50	150
TOTAL	800	850	400	1200

NAVIE BAYES CLASSIFIER ALGORITHM

- Applying Bayes' Theorem:
- $P(A|B)=[P(B|A) \ . P(A)]/P(B)$
- **P(Long | Orange) = [P(Orange | Long) * P(Long)] / P(Orange)**

- $[\left(\frac{0}{400}\right) * (\frac{400}{1200})]/(\frac{650}{1200}) = 0$

- So, **P(Long | Orange) = 0**

FRUIT	YELLOW	SWEET	LONG	TOTAL AVAILABLE FRUIT
ORANGE	350	450	0	650
BANANA	400	300	350	400
OTHERS	50	100	50	150
TOTAL	800	850	400	1200

NAVIE BAYES CLASSIFIER ALGORITHM

- **PROBABILTY CHART**

FRUIT	YELLOW	SWEET	LONG	TOTAL PROBABILTY
ORANGE	0.53	0.69	0	0
BANANA	1	0.75	0.88	0.66
OTHERS	0.33	0.67	0.33	0.073

- As per above chart, fruit **BANANA** is choosed based on the given attributes.

NAVIE BAYES CLASSIFIER ALGORITHM

- **NAVIE BAYES ALGORITHM : EXAMPLE TO SOLVE**
- **Problem: Identify the probability of the FRUIT which will be choose based on the given attributes**
- **Consider the below dataset:**

FRUIT	YELLOW	SWEET	LONG	TOTAL AVAILABLE FRUIT
ORANGE	200	400	0	550
BANANA	350	350	250	300
OTHERS	50	50	50	100
TOTAL	600	800	300	950

NAVIE BAYES CLASSIFIER ALGORITHM

- **PROBABILTY CHART**

FRUIT	YELLOW	SWEET	LONG	TOTAL PROBABILTY
ORANGE	0.36	0.73	0	0
BANANA	1.17	1.17	0.83	1.13
OTHERS	0.5	0.5	0.5	0.13

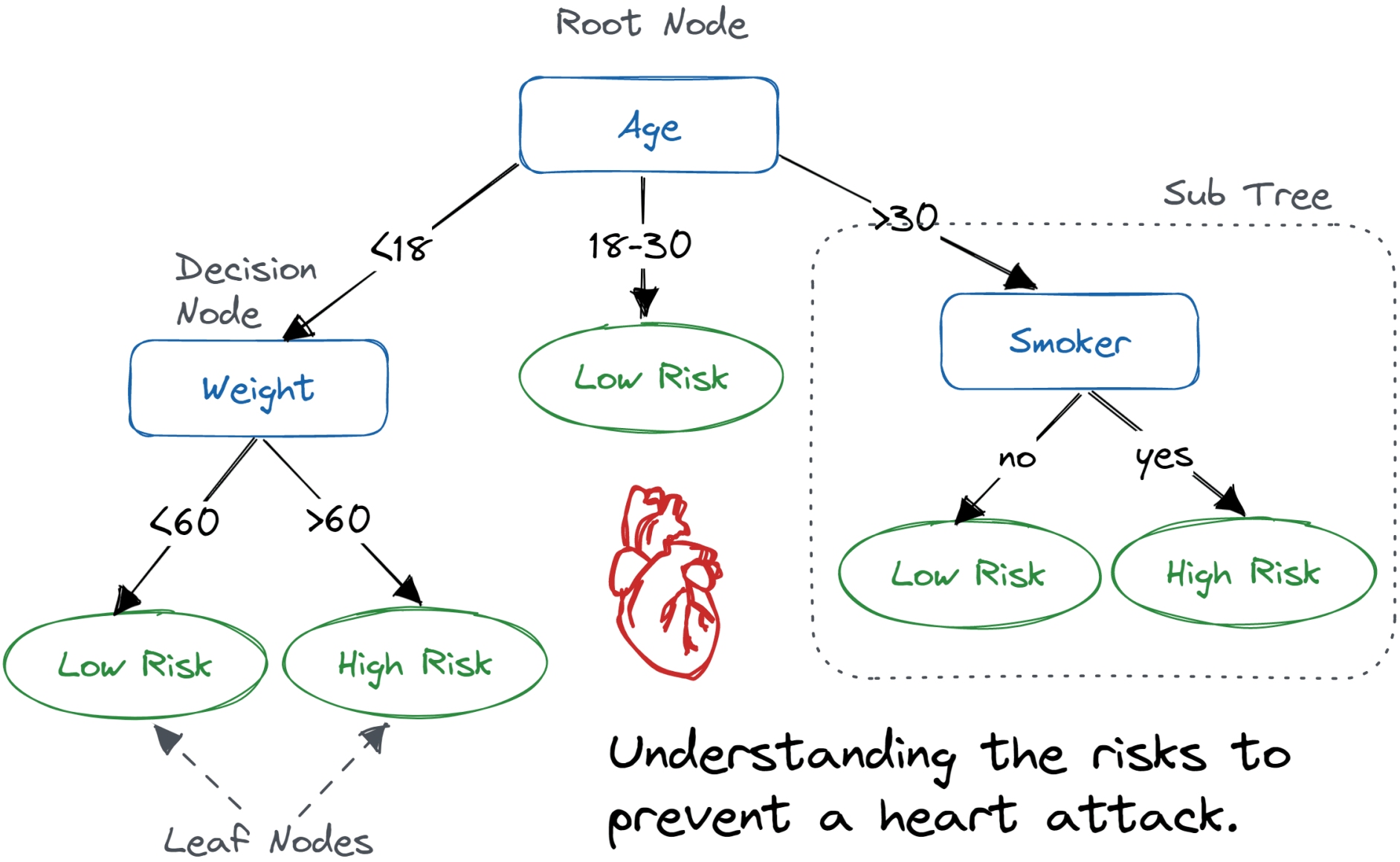
- As per above chart, fruit **BANANA** is choosed based on the given attributes.

DECISION TREE ALGORITHM

DECISION TREE ALGORITM

- A **decision tree** is a **supervised learning algorithm** used for both **classification** and **regression** tasks.

- It models decisions as a **tree-like structure** where,
- **internal nodes** represent **attribute tests**,
- **branches** represent **attribute values**, and
- **leaf nodes** represent **final decisions** or **predictions**.



Understanding the risks to prevent a heart attack.

DECISION TREE ALGORITM

- **WORKING OF DECISION TREE**
- **Step 1 – Ask a Question (Root Node):**
- Is it raining?
- If yes, you might decide to buy an umbrella.
- If no, you move to the next question.

DECISION TREE ALGORITM

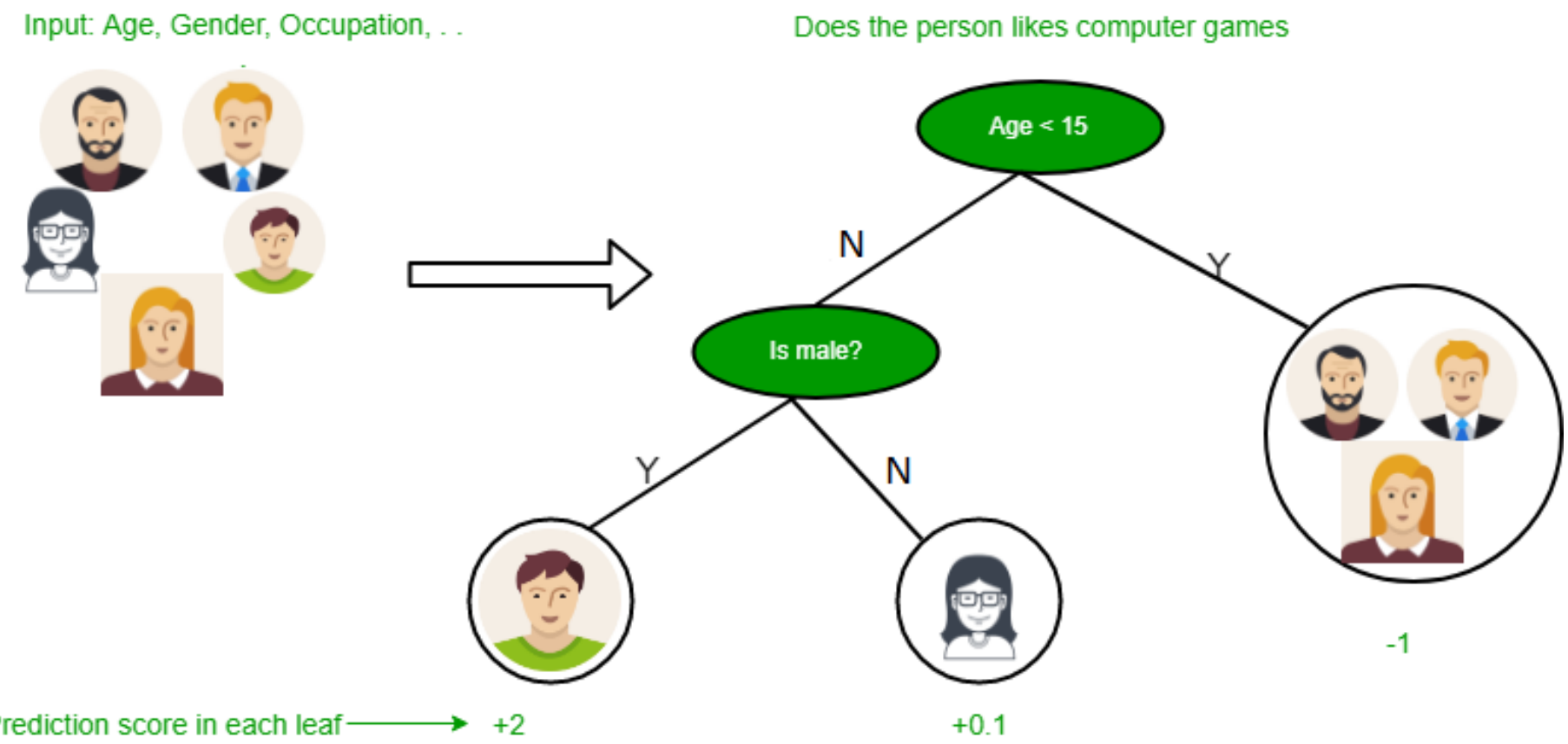
- **WORKING OF DECISION TREE**
- **Step 2 – More Questions (Internal Nodes):**
- If it’s not raining, you might ask:
- Is it likely to rain later?
- If yes, you buy an umbrella; if no, you don’t.

DECISION TREE ALGORITM

- **WORKING OF DECISION TREE**
- **Step 3 – Decision (Leaf Node):**
- Based on your answers, you either buy or skip the umbrella

DECISION TREE ALGORITHM EXAMPLE

DECISION TREE ALGORITM



DECISION TREE ALGORITM

- **Example: Predicting Whether a Person Likes Computer Games**
- Imagine you want to predict if a person enjoys computer games based on their age and gender.
- Here's how the decision tree works:
- **STEP-I**
- **Start with the Root Question (Age):**
- The first question is: "**Is the person's age less than 15?"**
- If **Yes**, move to the **left**.
- If **No**, move to the **right**.

DECISION TREE ALGORITM

- **Example: Predicting Whether a Person Likes Computer Games**

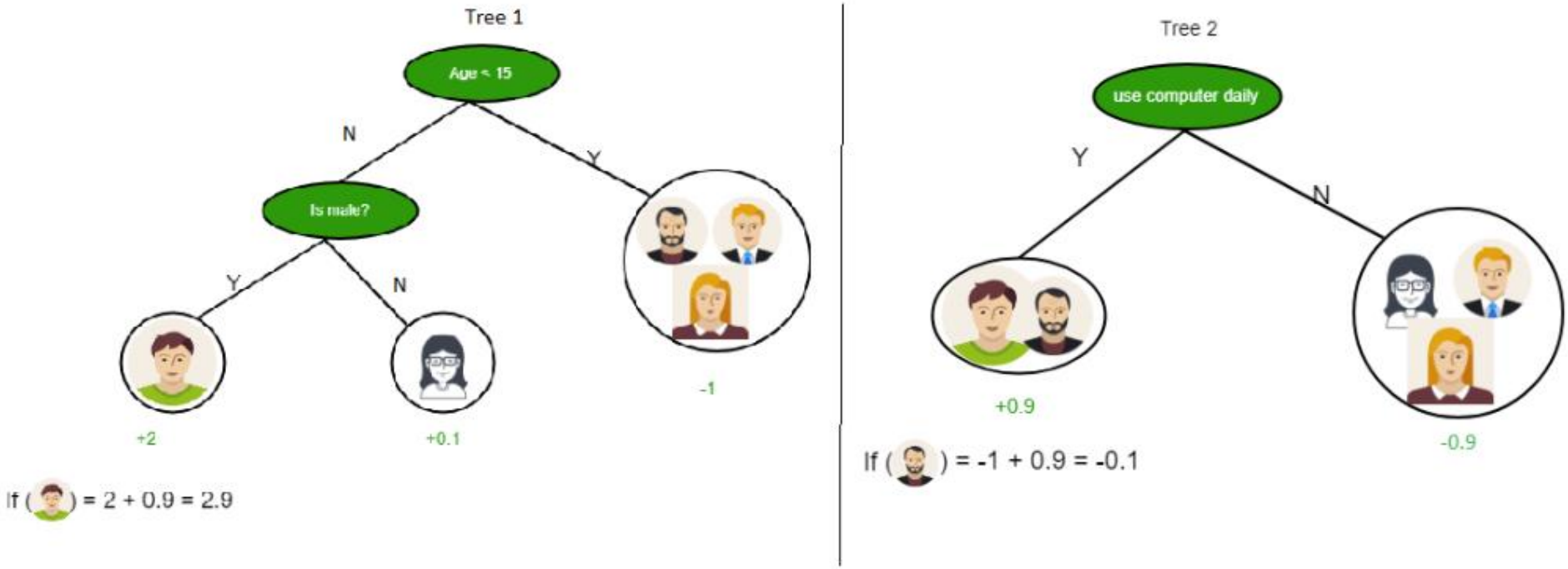
- **STEP-II**
- **Branch Based on Age:**
- If the person is **younger** than **15**, they are likely to enjoy **computer games (+2 prediction score**).
- If the person is **15** or **older**, ask the next question: "**Is the person male?"**

DECISION TREE ALGORITM

- **Example: Predicting Whether a Person Likes Computer Games**

- **STEP-III**
- **Branch Based on Gender (For Age 15+):**
- If the person is **male**, they are somewhat likely to enjoy computer games **(+0.1 prediction score**).
- If the person is **not male**, they are less likely to enjoy computer games **(-1 prediction score)**

DECISION TREE ALGORITM



DECISION TREE ALGORITM

- **Example: Predicting Whether a Person Likes Computer Games**

- **TREE 1: AGE AND GENDER**
- The first tree asks two questions:
- **“Is the person’s age less than 15?”**
  - If **Yes**, they get a score of **+2**.
  - If **No**, proceed to the **next question**.

- **“Is the person male?”**
  - If **Yes**, they get a score of **+0.1**.
  - If **No**, they get a score of **-1**.

DECISION TREE ALGORITM

- **Example: Predicting Whether a Person Likes Computer Games**

- **TREE 2: COMPUTER USAGE**
- The second tree focuses on daily computer usage:
- **“Does the person use a computer daily?”**
  - If **Yes**, they get a score of **+0.9**.
  - If **No**, they get a score of **-0.9**.

RANDOM FOREST ALGORITHM

RANDOM FOREST ALGORITM

- **Random Forest** is the **ensemble learning method**, which consists of a **large number** of **decision trees**.

- Each **decision tree** in a **random forest predicts** an **outcome**, and the **prediction** with the **majority** of **votes** is considered as the **outcome**.

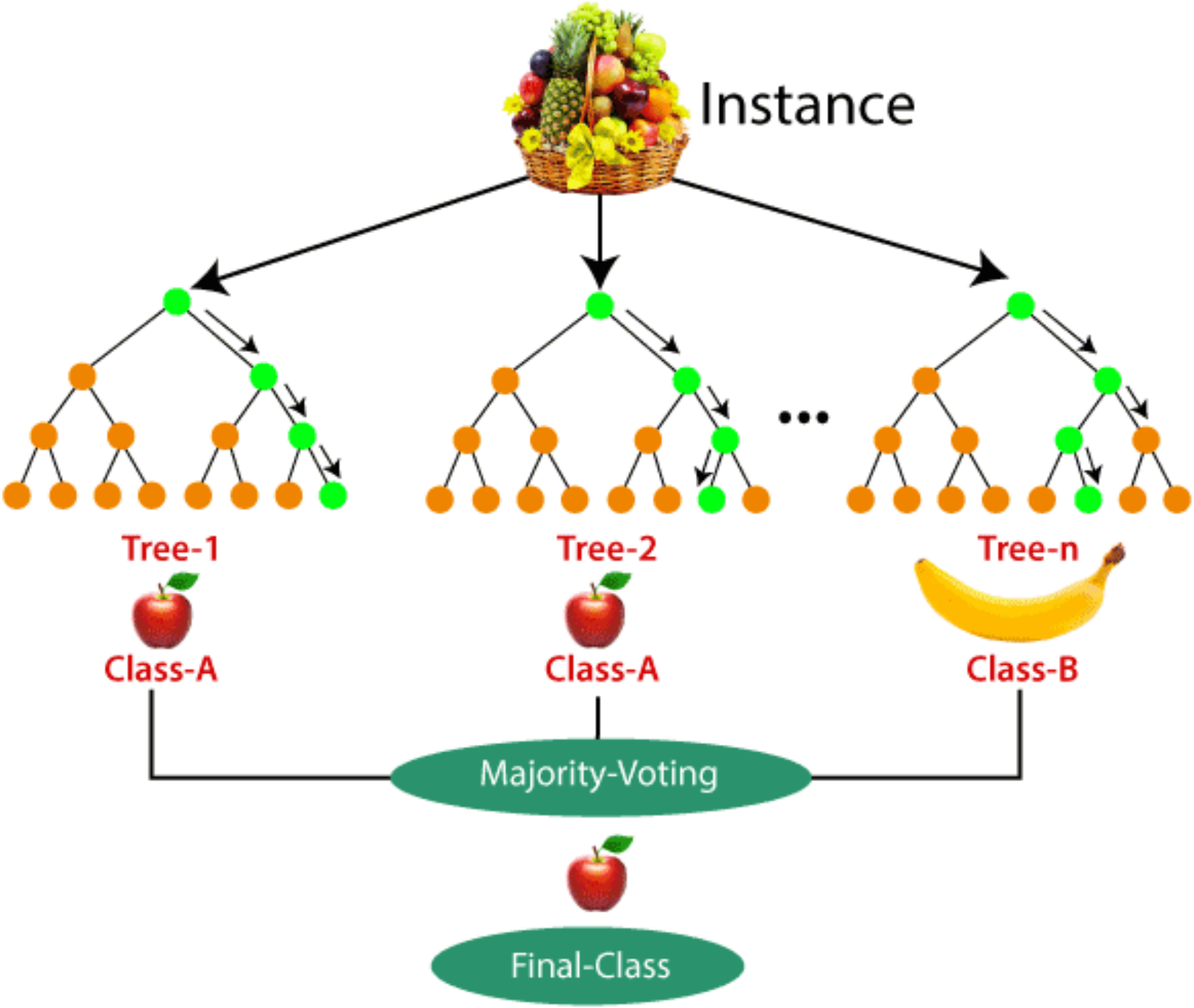
- A **random forest model** can be used for both **regression** and **classification** problems.

RANDOM FOREST ALGORITM

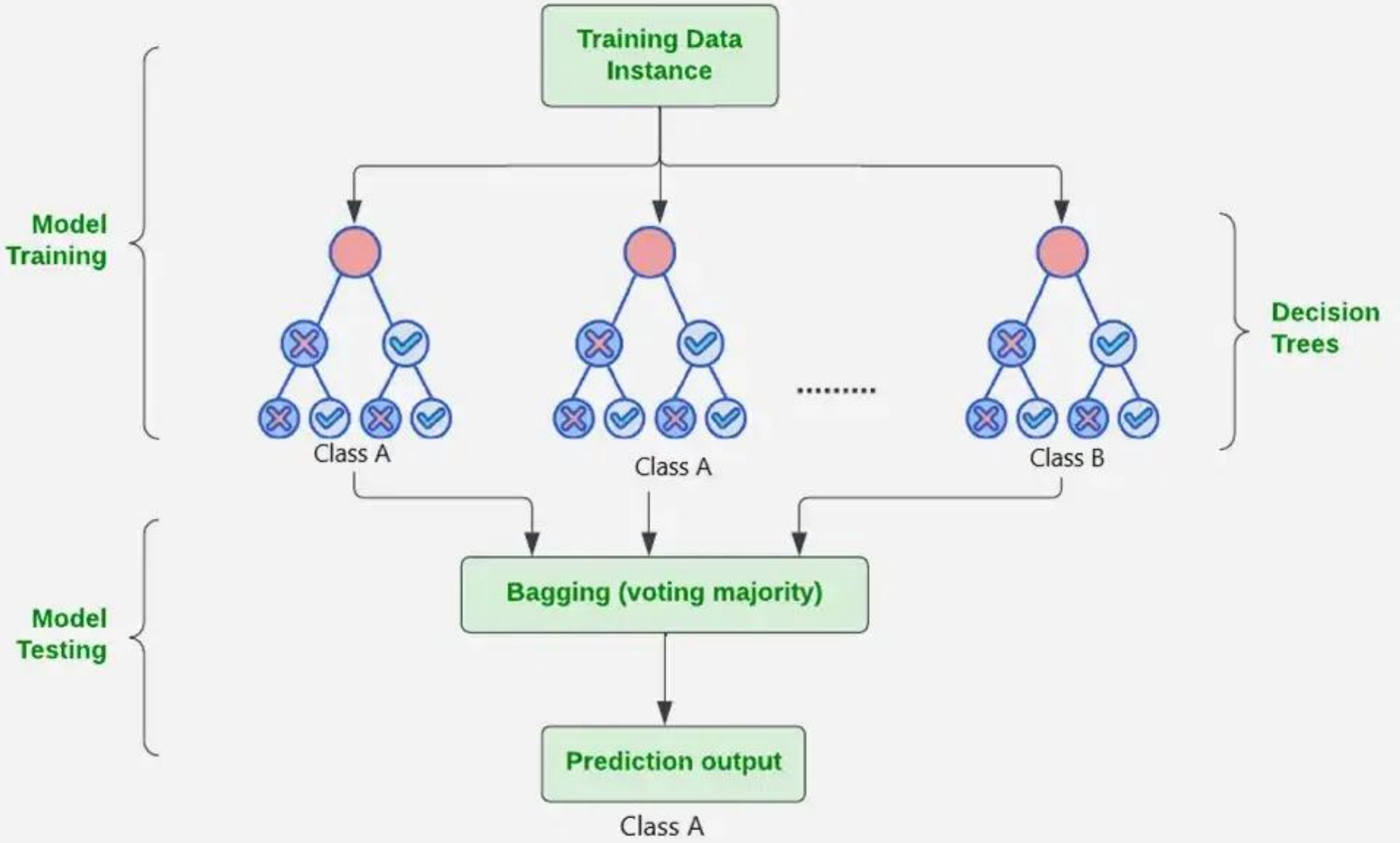
- It is a type of **classifier** that uses many **decision trees** to make **predictions**.

- It takes different **random parts** of the **dataset** to **train** each **tree** and then it **combines** the results by **averaging** them.

- This approach helps improve the **accuracy** of **predictions**.


Instance
Tree-1
Tree-2
Tree-n
Class-A
Class-A
Class-B
Majority-Voting
Final-Class

Random Forest Algorithm in Machine Learning



RANDOM FOREST ALGORITM

- **WORKING OF RANDOM FOREST**
- **Random Forest** builds **multiple decision trees** using **random samples** of the data.
- Each **tree** is trained on a **different subset** of the data which makes each **tree unique**.
- When creating each **tree** the **algorithm** randomly selects a **subset** of **features** or **variables** to **split** the **data** rather than using all **available features** at a time. This adds **diversity** to the trees.

RANDOM FOREST ALGORITM

- **WORKING OF RANDOM FOREST**
- Each **decision tree** in the **forest** makes a **prediction** based on the **data** it was **trained** on. When making **final prediction random forest** combines the **results** from all the **trees**.
- For **classification tasks** the **final prediction** is decided by a **majority vote**.
- For **regression tasks** the **final prediction** is the **average** of the **predictions** from all the **trees**.

RANDOM FOREST ALGORITHM EXAMPLE

RANDOM FOREST ALGORITM

- **ORIGINAL DATASET**

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	137	Yes

RANDOM FOREST ALGORITM

- **BOOTSTRAP DATASET**
- *Bootstrap Dataset is a dataset of the same size as the original, we just randomly select samples from the original dataset*.

RANDOM FOREST ALGORITM

- **ORIGINAL DATASET**

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	137	Yes

BOOTSTRAP DATASET

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	137	Yes
Yes	Yes	No	210	No

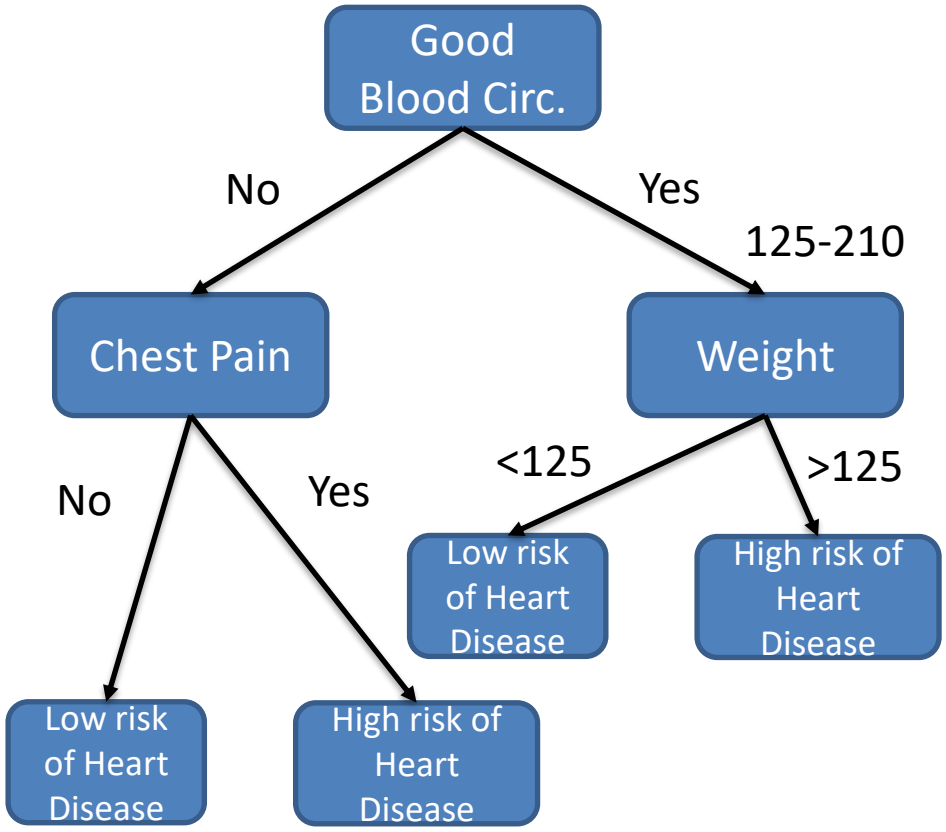
RANDOM FOREST ALGORITM

- **BOOTSTRAP DATASET**
- *Create a decision tree using Bootstrap dataset, but only use a random subset of variables (or columns) at each step.*

RANDOM FOREST ALGORITM

- **BOOTSTRAP DATASET**

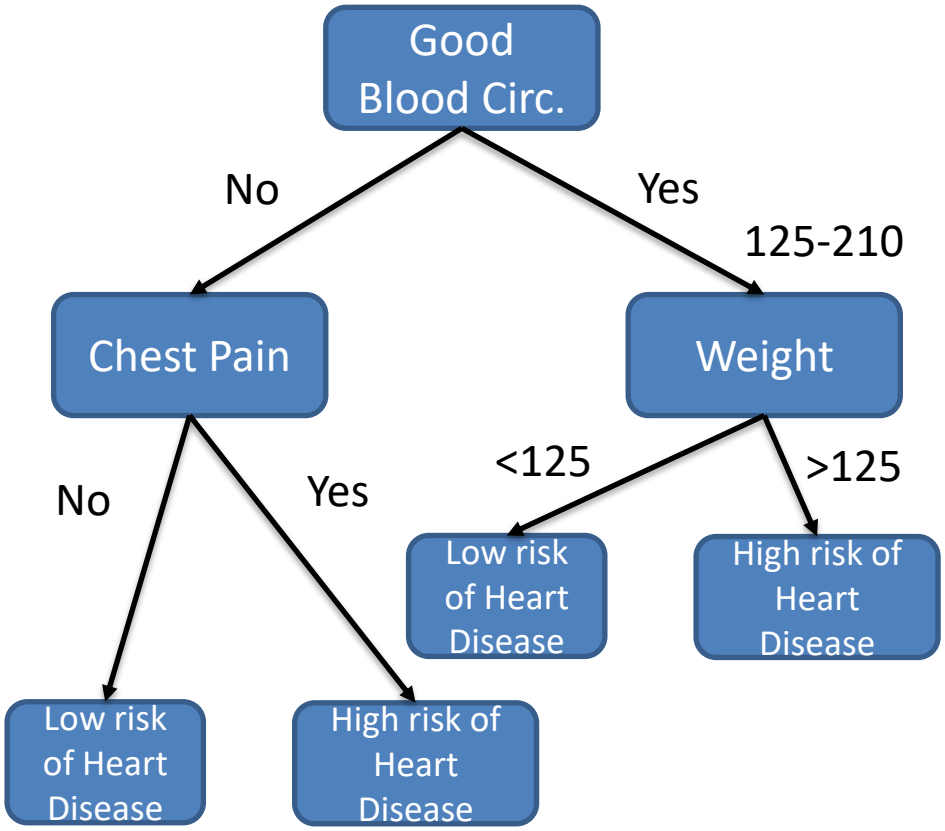
Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	137	Yes
Yes	Yes	No	210	No



RANDOM FOREST ALGORITM

- **BOOTSTRAP DATASET**

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	**YES**
No	No	No	125	**NO**
Yes	No	Yes	137	**YES**
Yes	Yes	No	210	**YES**



SUPPORT VECTOR MACHINE (SVM) ALGORITHM

SUPPORT VECTOR MACHINE

- **INTRODUCTION**
- **Support Vector Machine (SVM)** is one of the most popular **Supervised Learning algorithms**, which is used for **Classification** as well as **Regression** problems. However, primarily, it is used for **Classification** problems in Machine Learning.
- The goal of the SVM algorithm is to create the **best line** or **decision boundary** that can segregate **n-dimensional space** into **classes** so that we can easily put the **new data point** in the correct category in the future.
- This best decision boundary is called a **hyperplane**.

SUPPORT VECTOR MACHINE

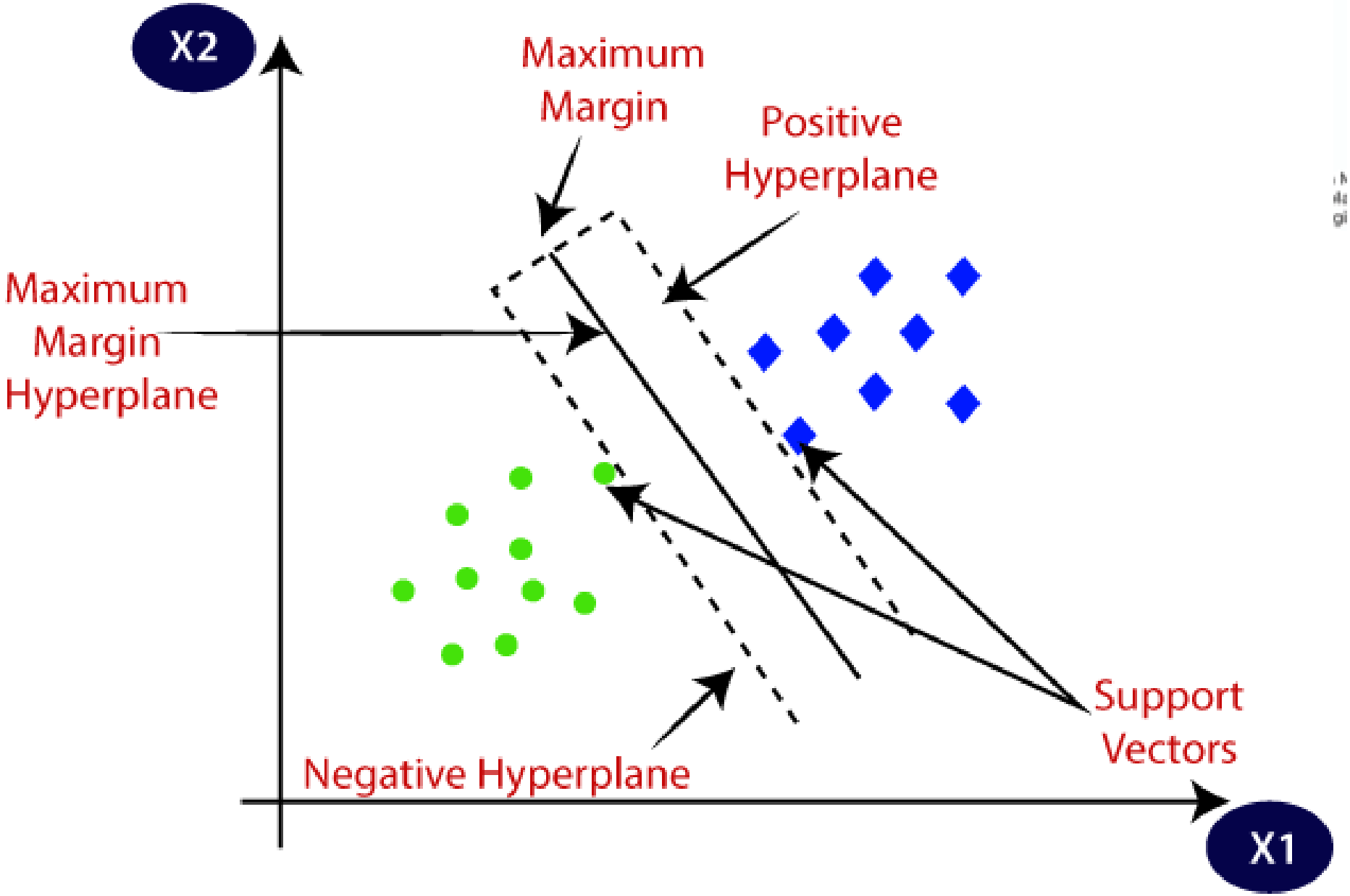
- **TYPES OF SVM**
- **Linear SVM:** Linear SVM is used for **linearly** separable data, which means if a **dataset** can be classified into **two classes** by using a **single straight line**, then such data is termed as linearly separable data, and classifier is used called as **Linear SVM classifier**.

- **Non-linear SVM:** Non-Linear SVM is used for **non-linearly** separated data, which means if a **dataset** cannot be classified by using a **straight line**, then such data is termed as non-linear data and classifier used is called as **Non-linear SVM classifier**.

SUPPORT VECTOR MACHINE

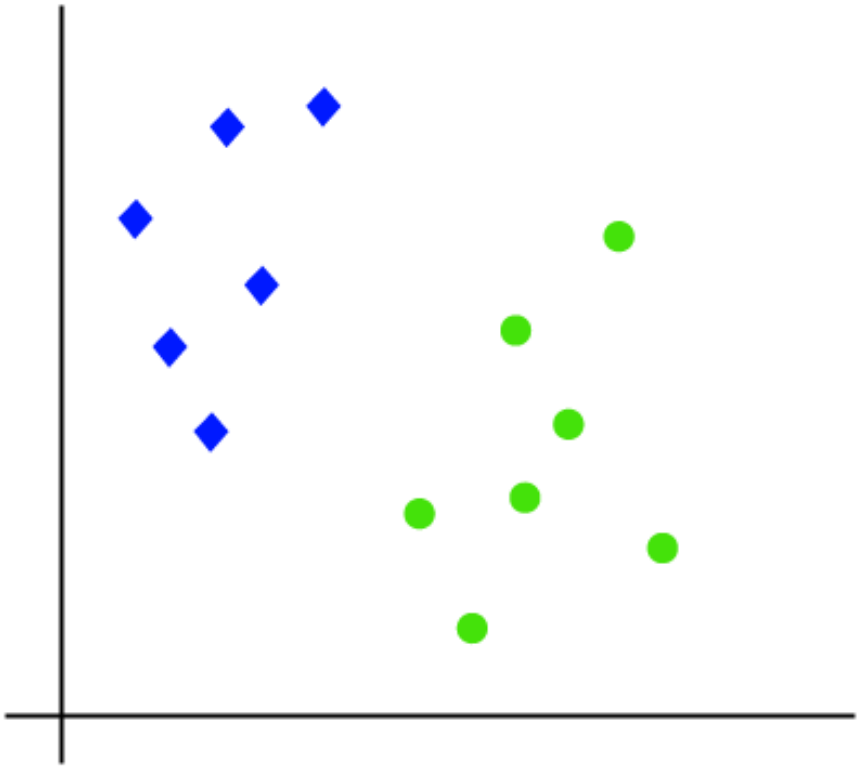
- **HYPERPLANE IN SVM**
- **Hyperplane:** There can be multiple lines/decision boundaries to segregate the classes in **n-dimensional space**, but we need to find out the **best decision boundary** that helps to classify the data points. This best boundary is known as the hyperplane of SVM.

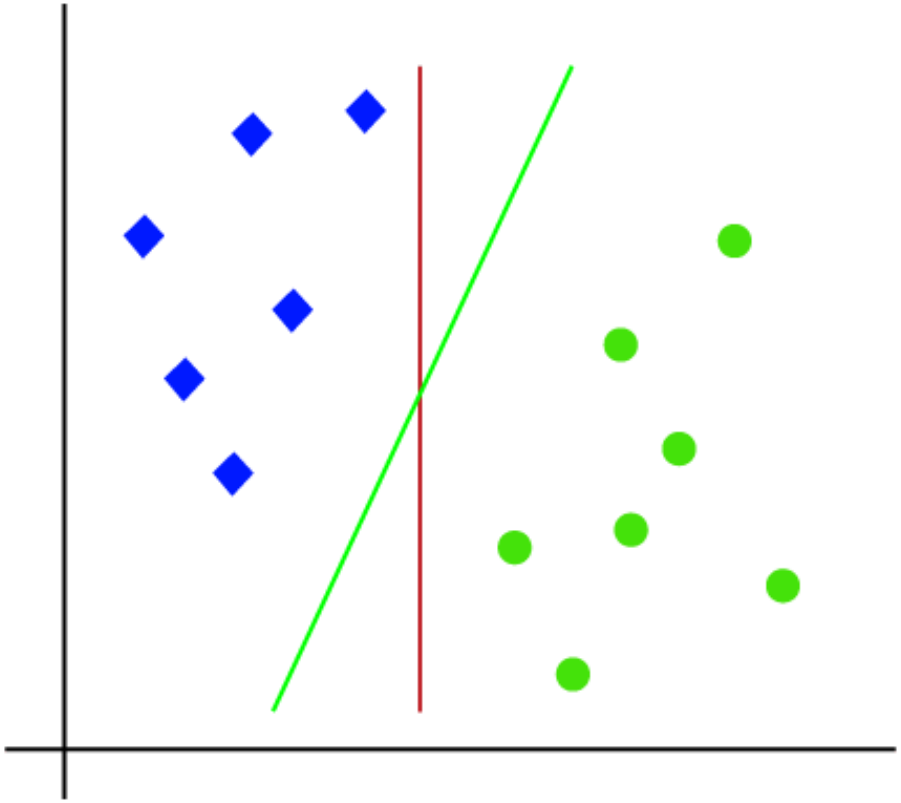
- The dimensions of the hyperplane depend on the features present in the dataset, which means if there are **2 features** then **hyperplane** will be a **straight line**. And if there are **3 features**, then **hyperplane** will be a **2-dimension plane**.


X2
Maximum
Margin
Positive
Hyperplane
Maximum
Margin
Hyperplane
Support
Vectors
Negative Hyperplane
X1

SUPPORT VECTOR MACHINE

- **WORKING OF LINEAR SVM**



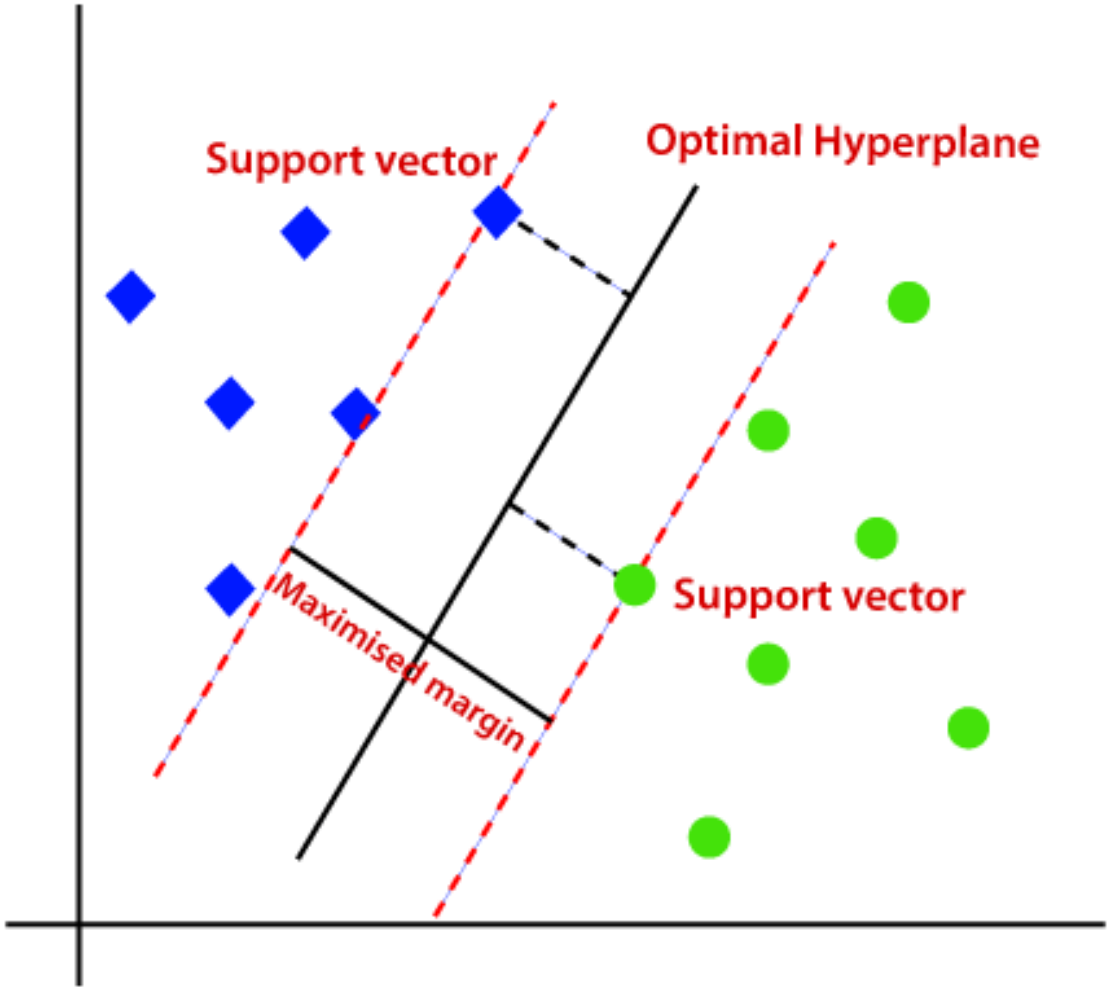


SUPPORT VECTOR MACHINE

- **WORKING OF LINEAR SVM**
- The SVM algorithm helps to find the **best line** or **decision boundary**; this best boundary or region is called as a **hyperplane**.
- SVM algorithm finds the **closest point** of the **lines** from both the **classes**. These points are called **support vectors**.
- The **distance** between the **vectors** and the **hyperplane** is called as **margin**. And the goal of SVM is to **maximize** this **margin**.
- The **hyperplane** with **maximum margin** is called the **optimal hyperplane**.

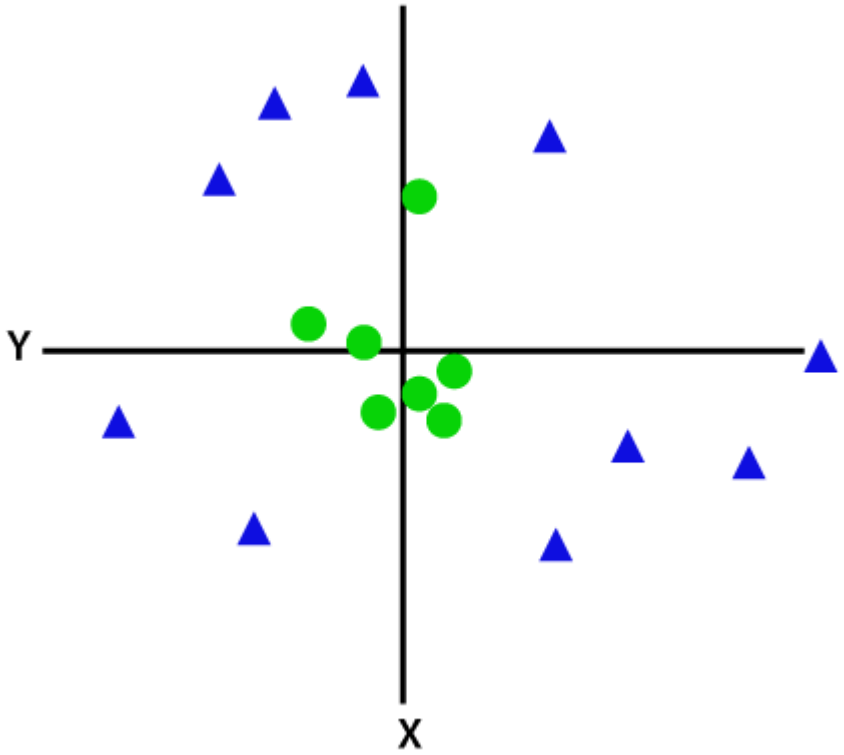
SUPPORT VECTOR MACHINE

- **WORKING OF LINEAR SVM**



SUPPORT VECTOR MACHINE

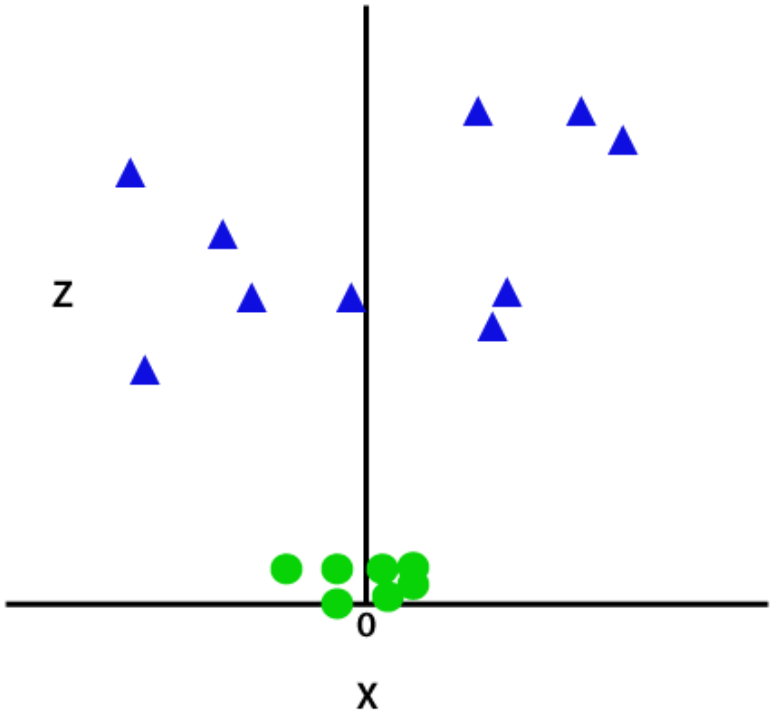
- **WORKING OF NON-LINEAR SVM**
- If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line.



SUPPORT VECTOR MACHINE

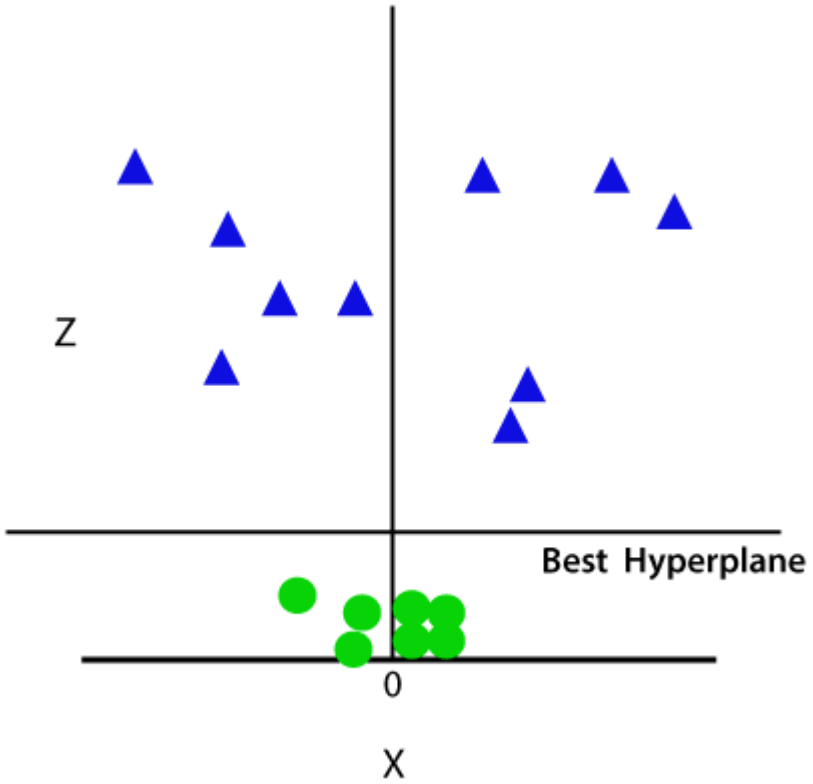
- **WORKING OF NON-LINEAR SVM**
- So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z.

$$z=x^2 + y^2$$



SUPPORT VECTOR MACHINE

- **WORKING OF NON-LINEAR SVM**
- So now, SVM will divide the datasets into classes in the following way. Consider the below image:



MODEL EVALUATION

BIAS AND VARIANCE

BIAS AND VARIANCE

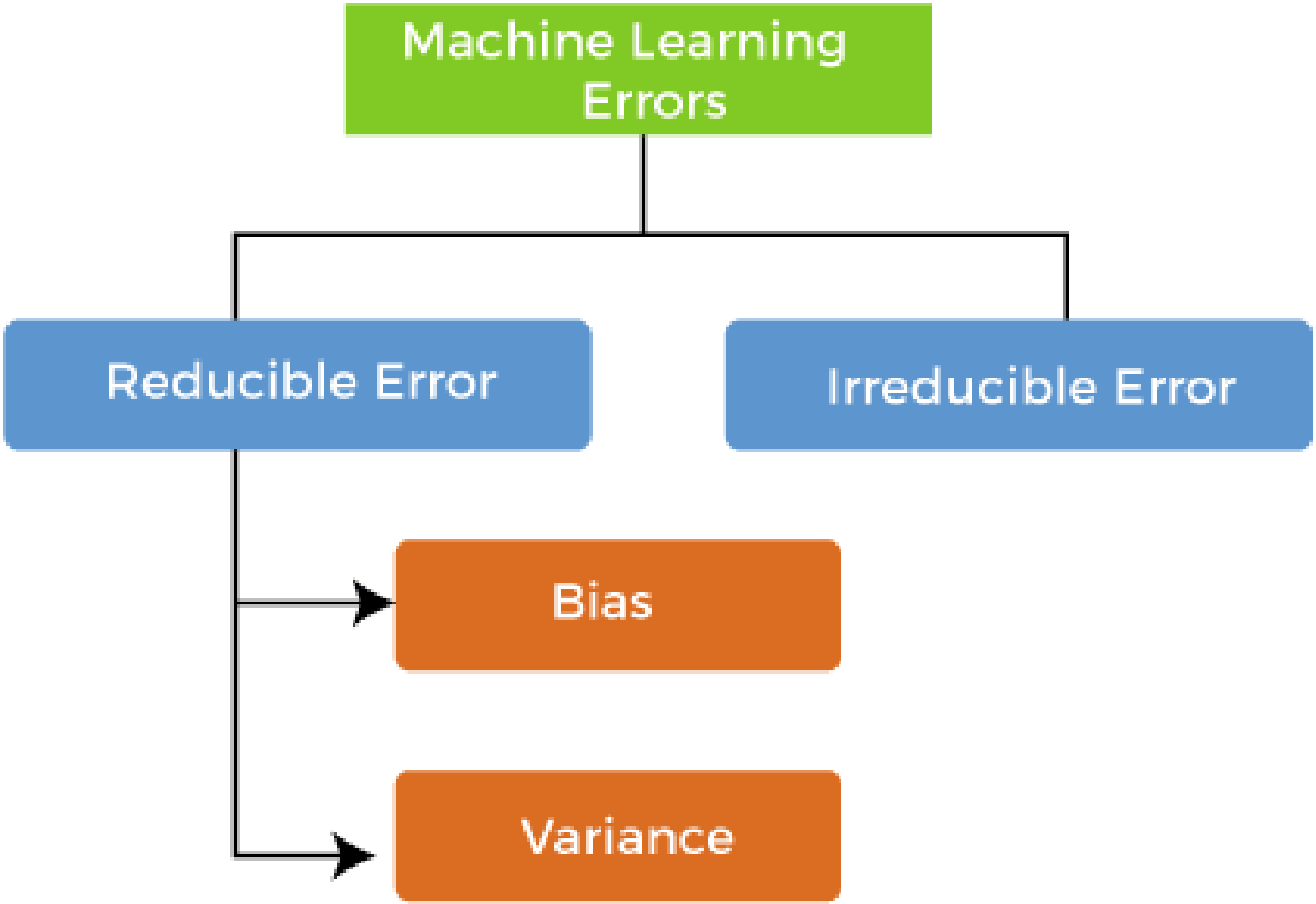
- Machine learning is a branch of Artificial Intelligence, which allows machines to perform **data analysis** and make **predictions**.
- However, if the machine learning model is **not accurate**, it can make **predictions errors**, and these **prediction errors** are usually known as **Bias and Variance**.
- In machine learning, these **errors** will always be present as there is always a **slight difference** between the **model predictions** and **actual predictions**.

BIAS AND VARIANCE

- **ERRORS IN MACHINE LEARNING?**
- ***In machine learning, an error is a measure of how accurately an algorithm can make predictions for the previously unknown dataset.***

- On the basis of these errors, the machine learning model is selected that can perform best on the particular dataset.

BIAS AND VARIANCE



BIAS AND VARIANCE

- **TYPES OF ERRORS**
- **Reducible errors:** These errors can be reduced to improve the **model accuracy**. Such errors can further be classified into **bias** and **Variance**.
- **Irreducible errors:** These errors will always be present in the model regardless of which algorithm has been used.
- The cause of these errors is unknown variables whose value can't be reduced.

BIAS AND VARIANCE

- **WHAT IS BIAS?**
- In general, a machine learning model **analyses the data**, find **patterns** in it and make **predictions**.
- While training, the model learns these **patterns** in the **dataset** and applies them to **test data** for **prediction**.
- While making predictions, a **difference** occurs between **prediction values** made by the model and **actual /expected values**, and this difference is known as **bias errors** or **Errors due to bias**.

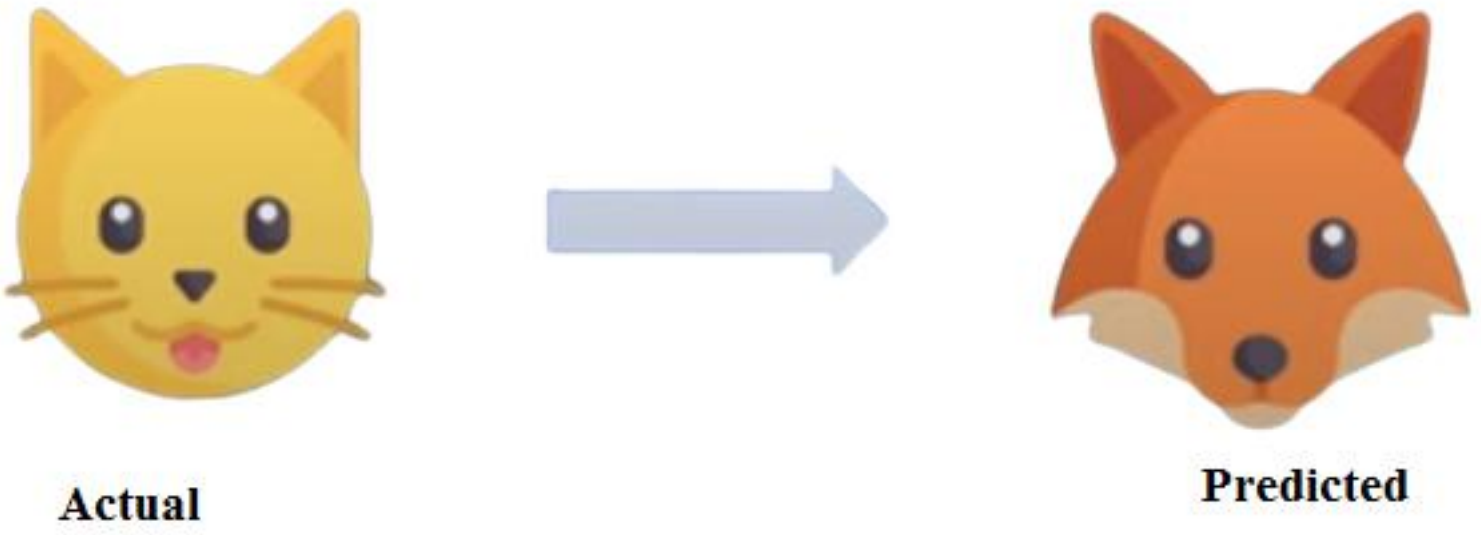
BIAS AND VARIANCE

- A model has either:
- **Low Bias:** A low bias model will make **fewer assumptions** about the form of the **target function**.

- **High Bias:** A model with a **high bias** makes more **assumptions**, and the model becomes unable to capture the important **features** of our **dataset**.
- A **high bias model** also ***cannot perform well on new data***.

BIAS AND VARIANCE

- Suppose we want our model to **predict** the **animal** by showing photos of animals. We **trained** the **model** on only **one attribute pointing ears**. Then we showed the image of a **cat** to the **model**. So, the **model predicted** it as a **fox** also has **pointed ears**.



BIAS AND VARIANCE

- **WHAT IS VARIANCE?**
- The variance would specify the **amount** of **variation** in the **prediction** if the **different training data** was used.
- In simple words, variance tells that how much a **random variable** is **different** from its **expected value**.
- Variance errors are either of **low variance** or **high variance**.

BIAS AND VARIANCE

- **Low variance** means there is a **small variation** in the **prediction** of the **target function** with changes in the **training data set**.

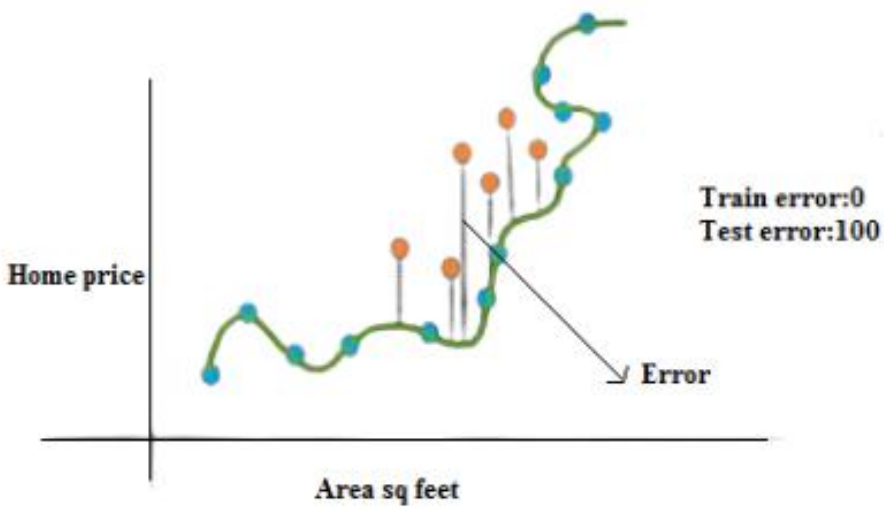
- **High variance** shows a **large variation** in the **prediction** of the **target function** with changes in the **training dataset**.

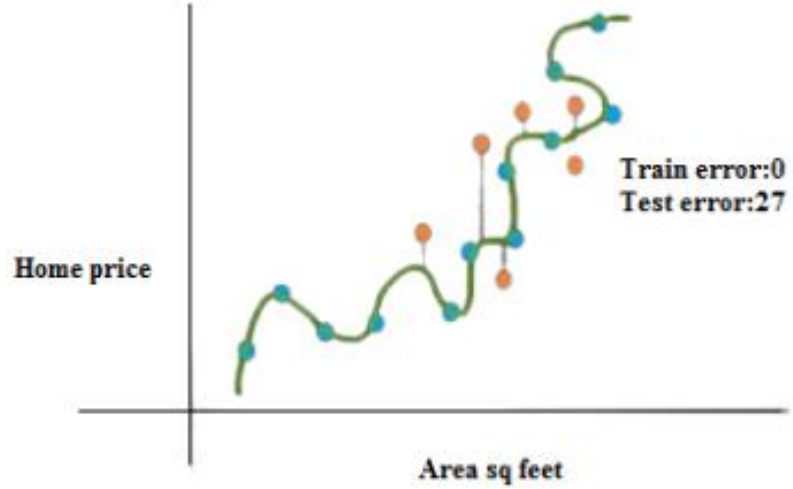
BIAS AND VARIANCE

- Consider the **blue dots** are **training samples**, the **orange dots** are **test samples**. We can train a model that fits these **blue dots** perfectly which means our model is an **overfitted model**.

- *An overfit model tries to fit exactly to the training samples but not to the test samples that's why training error becomes close to zero and test error is high.*

BIAS AND VARIANCE





Test error=100 ⟷ High variance ⟷ Test error=27

CONFUSION MATRIX

CONFUSION MATRIX

- A **confusion matrix** is a **simple table** that shows how well a **classification model** is performing by comparing its **predictions** to the **actual results**.
- It breaks down the **predictions** into **four categories**: **correct predictions** for both **classes** (*true positives and true negatives*) and **incorrect predictions** (*false positives and false negatives*).
- This helps you understand where the model is making **mistakes**, so you can **improve** it.

CONFUSION MATRIX

- **True Positive (TP):** The model correctly predicted a positive outcome (the actual outcome was positive).
- **True Negative (TN):** The model correctly predicted a negative outcome (the actual outcome was negative).
- **False Positive (FP):** The model incorrectly predicted a positive outcome (the actual outcome was negative). Also known as a Type I error.
- **False Negative (FN):** The model incorrectly predicted a negative outcome (the actual outcome was positive). Also known as a Type II error.

CONFUSION MATRIX

- **EXAMPLE: A 2X2 Confusion matrix is shown below for the image recognition having a Dog image or Not Dog image**
- **True Positive (TP):** It is the total counts having both **predicted** and **actual values** are **Dog**.
- **True Negative (TN):** It is the total counts having both **predicted** and **actual values** are **Not Dog**.
- **False Positive (FP):** It is the total counts having **prediction** is **Dog** while actually **Not Dog**.
- **False Negative (FN):** It is the total counts having **prediction** is **Not Dog** while actually, it is **Dog**.

CONFUSION MATRIX

	Predicted	Predicted
Actual	True Positive (TP)	False Negative (FN)
Actual	False Positive (FP)	True Negative (TN)

Index	1	2	3	4	5	6	7	8	9	10
Actual	Dog	Dog	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Not Dog	Not Dog
Predicted	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Dog	Dog	Not Dog	Not Dog
Result	TP	FN	TP	TN	TP	FP	TP	TP	TN	TN

CONFUSION MATRIX

- Actual Dog Counts = 6
- Actual Not Dog Counts = 4
- True Positive Counts = 5
- False Positive Counts = 1
- True Negative Counts = 3
- False Negative Counts = 1

		Predicted	
		Dog	**Not Dog**
Actual	**Dog**	True Positive (TP =5)	False Negative (FN =1)
	Not Dog	False Positive (FP=1)	True Negative (TN=3)

ACCURACY

ACCURACY

- **Accuracy** measures how often the **model's predictions** are **correct overall**.
- It gives a general idea of how well the model is **performing**.
- However, **accuracy** can be **misleading**, especially with **imbalanced datasets** where **one class dominates**.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

PRECISION

PRECISION

- **Precision** focuses on the **quality** of the model's **positive predictions**.
- It tells us how many of the **instances predicted** as **positive** are actually **positive**.
- **Precision** is important in situations where **false positives** need to be **minimized**, such as detecting **spam emails** or **fraud**.

$$\text{Precision} = \frac{TP}{TP+FP}$$

RECALL

RECALL

- **Recall** measures how well the **model** identifies all **actual positive cases**.
- It shows the **proportion** of **true positives** detected out of all the **actual positive instances**.
- **High recall** is essential when **missing positive cases** has significant consequences, such as in **medical diagnoses.**

$$\text{Recall} = \frac{TP}{TP+FN}$$

F1-Score

F1-SCORE

- **F1-score** combines **precision** and **recall** into a single metric to balance their **trade-off**. It provides a better sense of a model's overall **performance**, particularly for **imbalanced datasets**.
- The **F1 score** is helpful when both **false positives** and **false negatives** are important, though it assumes **precision** and **recall** are equally significant, which might not always align with the use case.

$$\text{F1-Score} = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$